

Rapport de Projet de Fin d'Études

Présenté en vue de l'obtention du

Diplôme National d'Ingénieur en Sciences Appliquées et Technologiques

Spécialité : ingénierie des systèmes embarqués et objets connectés

Réalisé par

Adem Yahya

Pipeline de traitement d'images embarqué pour ADAS sur FPGA et GPU

Encadrant professionnel : Mohamed Taib

Ingénieur R&D

Encadrante académique : Zouhour Ben Azouz

Maître Assistante

Réalisé au sein de Capgemini



Rapport de Projet de Fin d'Études

Présenté en vue de l'obtention du

Diplôme National d'Ingénieur en Sciences Appliquées et Technologiques

Spécialité : ingénierie des systèmes embarqués et objets connectés

Par

Adem Yahya

Pipeline de traitement d'images embarqué pour ADAS sur FPGA et GPU

Encadrant professionnel : **Mohamed Taib**

Ingénieur R&D

Encadrant académique : **Zouhour Ben Azouz**

Maître Assistante

Réalisé au sein de Capgemini



J'autorise l'étudiant à faire le dépôt de son rapport de stage en vue d'une soutenance.

Encadrant professionnel, **Monsieur, Mohamed Taib**

Signature et cachet

J'autorise l'étudiant à faire le dépôt de son rapport de stage en vue d'une soutenance.

Encadrante académique, **Madame, Zouhour Ben Azouz**

Signature

Dédicaces

Je dédie ce travail à :

À ma mère, pour son amour et son soutien inestimables.

À mon père, pour sa confiance et ses encouragements.

À ma famille, pour son appui et sa présence à chaque étape de ma vie.

À mes amis, pour leur soutien et les moments de joie partagés etc.

Adem Yahya

Remerciements

Mes remerciements vont également à l'entreprise Capgemini pour m'avoir offert l'opportunité d'effectuer ce stage dans un cadre professionnel enrichissant. Je remercie tout particulièrement M. Mohamed Taïbe, mon directeur de projet, pour ses précieux conseils, son soutien constant et sa confiance tout au long de ce travail. J'exprime aussi ma reconnaissance à mon encadrante académique Ms Zouhour Ben Azouz pour son suivi, ses conseils avisés et son accompagnement pédagogique. Enfin, je remercie l'ensemble de mes professeurs, qui ont contribué à ma formation et m'ont transmis le savoir nécessaire pour mener à bien ce projet.

Table des matières

Introduction générale	1
1 Contexte général	2
1.1 Organisme d'accueil	3
1.2 Problématique	4
1.3 Étude et critique de l'existant	4
1.4 Cahier des charges	6
1.5 Choix méthodologique	7
1.5.1 Application de la méthodologie PDCA	8
1.5.2 Diagramme de Gantt	9
2 État de l'art	10
2.1 Systèmes ADAS : définition et architecture	11
2.1.1 Définition et enjeux	11
2.1.2 Architecture sensorielle	12
2.1.3 Fonctions principales	13
2.1.4 Pipeline de traitement d'image	14
2.2 Modèles de détection d'objets	14
2.2.1 Détecteurs à deux étapes (Two-Stage Detectors)	15
2.2.2 Détecteurs à une étape (One-Stage Detectors)	15
2.2.3 La famille YOLO	16
2.2.4 Métriques d'évaluation	17
2.3 Techniques d'optimisation	18
2.4 Distillation de connaissances	19
2.4.1 Motivations et principe	20
2.4.2 Formulation mathématique	20
2.4.3 Taxonomie des méthodes de distillation	21
2.4.4 Distillation pour la détection d'objets	22
2.5 Datasets de référence pour la détection ADAS	22

2.5.1	KITTI	22
2.5.2	BDD100K	23
2.5.3	COCO (<i>Common Objects in Context</i>)	23
2.5.4	GTSDb (<i>German Traffic Sign Detection Benchmark</i>)	23
2.5.5	Europe Signs	24
2.6	Synthèse et justification des choix technologiques	24
2.6.1	Choix du modèle de détection : YOLOv8	24
2.6.2	Choix de la technique d'optimisation : distillation de connaissances	25
2.6.3	Stratégie de quantification pour le déploiement embarqué	26
2.6.4	Choix des datasets retenus : GTSDb et Europe Signs	27
2.6.5	Récapitulatif des choix	27
3	Conception et choix technologiques	29
3.1	Analyse des besoins	30
3.1.1	Besoins fonctionnels	30
3.1.2	Besoins non fonctionnels	31
3.1.3	Synthèse des besoins	31
3.2	Architecture globale du système	32
3.3	Architectures partielles par étape	33
3.3.1	Étape 1 : construction et enrichissement du dataset	33
3.3.2	Étape 2 : entraînement indépendant Teacher et Student	34
3.3.3	Étape 3 : distillation des connaissances	34
3.3.4	Étape 4a : déploiement sur Jetson Nano (FP16 / TensorRT)	35
3.3.5	Étape 4b : déploiement sur FPGA ZedBoard (INT8 / Vitis AI)	36
3.4	Choix technologiques	37
3.4.1	Choix matériels	37
3.4.2	Choix logiciels	39
3.4.3	Bibliothèques Python utilisées	40
4	Réalisation	43
4.1	Choix techniques	44
4.2	Plateformes embarquées pour l'inférence IA	44

4.2.1	Critères de sélection	44
4.2.2	GPU : NVIDIA Jetson Nano	44
4.2.3	FPGA : ZedBoard (Xilinx Zynq-7020)	45
4.2.4	PC de développement (GPU dédié)	46
4.2.5	Comparaison des plateformes cibles	47
4.3	Travail réalisé	47
4.4	Planning réel du projet	47
Conclusion générale		48
Bibliographie		49
Annexes		51
	Annexe 1. Exemple d'annexe	51
	Annexe 2. Entreprise	52

Table des figures

1.1	Logo Entreprise	3
1.2	Cycle PDCA	8
1.3	diagramme de Gantt	9
2.1	Niveaux d'automatisation SAE	12
2.2	Exemple adas VALEO[3]	13
2.3	Pipeline de traitement ADAS	14
2.4	exemple knowledge distillation	20
3.1	Architecture globale du système ADAS : pipeline de traitement d'image et de détection d'objets	32
3.2	Pipeline de construction et d'enrichissement du dataset ADAS	33
3.3	Entraînement indépendant : <i>teacher</i> YOLOv8m et <i>student</i> baseline YOLOv8n	34
3.4	Architecture de distillation des connaissances : YOLOv8m (<i>teacher</i>) → YOLOv8n (<i>student</i>)	35
3.5	Déploiement sur Jetson Nano avec optimisation FP16 / TensorRT	36
3.6	Déploiement sur ZedBoard FPGA avec quantification INT8 / Vitis AI	36
3.7	Carte NVIDIA Jetson Nano	38
3.8	carte FPGA zedboard	38
4.1	carte jetson nano	45
4.2	carte FPGA zedboard	46
Annexe 2.1	Logo d'entreprise	52

Liste des tableaux

1.1	Comparaison des solutions ADAS existantes	5
2.1	Évolution des versions YOLO et performances sur COCO val2017	17
2.2	Comparaison des architectures de détection pour le contexte ADAS embarqué	25
2.3	Comparaison des méthodes de distillation choix retenu pour YOLOv8	26
2.4	Impact de la quantification FP16 et INT8 sur YOLOv8 pour déploiement embarqué	26
2.5	Récapitulatif des choix technologiques retenus	28
4.1	Comparaison des plateformes cibles pour le déploiement du modèle ADAS	47
Annexe 1.1	Exemple tableau dans l'annexe	51

Liste des abréviations

- **GISI** = Génie Informatique des Systèmes Industriels
- **GLSI** = **G**énie **L**ogiciel et **S**ystèmes d'**I**nformation
- **GTR** = Génie des Télécommunications et Réseaux

Introduction générale

Le secteur automobile connaît aujourd’hui une transformation majeure grâce à l’intégration des systèmes intelligents et des techniques d’intelligence artificielle. Cette évolution a permis l’émergence des systèmes avancés d’aide à la conduite (ADAS), qui occupent désormais une place centrale dans le développement des véhicules modernes en améliorant la sécurité routière et le confort de conduite.

Ces systèmes reposent principalement sur des techniques de vision par ordinateur permettant d’analyser l’environnement du véhicule en temps réel, notamment la détection de véhicules, de piétons, de panneaux de signalisation et de marquages routiers. Les modèles d’intelligence artificielle qui les sous-tendent sont de plus en plus performants, mais également de plus en plus complexes, ce qui implique des besoins importants en puissance de calcul.

Dans ce contexte, le choix de l’architecture matérielle devient un enjeu crucial pour garantir un fonctionnement rapide, fiable et efficace sur des plateformes embarquées soumises à des contraintes strictes de mémoire et de consommation énergétique. Les GPU et les FPGA représentent deux solutions complémentaires pour l’accélération du traitement, chacune présentant des caractéristiques spécifiques en termes de parallélisme, de performance et d’efficacité énergétique.

Ce projet de fin d’études, réalisé au sein de Capgemini, s’inscrit dans cette dynamique. Il a pour objectif de concevoir un système de détection embarqué pour les applications ADAS, intégrant des techniques d’optimisation et de compression des modèles – notamment la distillation de connaissances et la quantification – afin de réduire leur complexité tout en préservant de bonnes performances de détection. Une étude comparative entre les architectures GPU et FPGA est également menée, en analysant les temps d’exécution, l’utilisation des ressources matérielles et l’efficacité énergétique, dans le but d’identifier la solution la plus adaptée aux exigences des systèmes ADAS embarqués.

Ce rapport est organisé en quatre chapitres. Le premier présente le contexte général du projet et la problématique posée. Le deuxième est consacré à l’état de l’art des technologies utilisées. Le troisième détaille la conception du système et les choix technologiques adoptés. Le quatrième présente la réalisation, l’optimisation du modèle et l’évaluation comparative des performances sur les plateformes GPU et FPGA.

Contexte général

Plan

1	Organisme d'accueil	3
2	Problématique	4
3	Étude et critique de l'existant	4
4	Cahier des charges	6
5	Choix méthodologique	7

Introduction

Dans ce chapitre présente le cadre général dans lequel s'inscrit ce projet de fin d'études. Nous commençons par décrire l'entreprise d'accueil et son domaine d'activité, puis nous exposons la problématique principale du projet. Une analyse des solutions existantes est ensuite réalisée, avant de définir les objectifs à atteindre et la démarche de travail adoptée.

1.1 Organisme d'accueil

Dans cette partie, nous présentons l'entreprise dans laquelle le stage a été effectué, en mettant en avant son secteur d'activité ainsi que son organisation.

Depuis sa fondation en 1967 en France, Capgemini s'est imposée comme un acteur majeur du conseil, des services numériques et de l'innovation technologique. L'entreprise s'appuie sur une expertise reconnue dans plusieurs domaines, notamment le développement logiciel, les systèmes embarqués, la cybersécurité, l'intelligence artificielle ainsi que l'Internet des objets (IoT). Son approche repose sur la combinaison d'un savoir-faire technique solide et d'une compréhension approfondie des problématiques métiers, ce qui lui permet de concevoir des solutions technologiques efficaces et durables. Présente dans plus de 50 pays, l'entreprise accompagne ses clients dans leur transformation digitale grâce à des solutions innovantes et adaptées à leurs besoins **capgemini2024**.

Dans le cadre de ce projet, le stage a été effectué au sein de la division Capgemini Engineering, plus précisément dans l'équipe Systèmes Embarqués et Automotive. Cette entité intervient sur des projets d'intégration de solutions d'intelligence artificielle embarquée pour des clients du secteur automobile, notamment dans les domaines de la perception, de la fusion de capteurs et du traitement temps réel. C'est dans ce contexte industriel que s'inscrit ce projet de fin d'études, qui vise à concevoir et évaluer un pipeline de détection d'objets embarqué destiné aux systèmes ADAS, comme illustré par le logo de la société dans la figure 1.1.



Figure 1.1 : Logo Entreprise

1.2 Problématique

Avec l'évolution rapide des technologies de l'intelligence artificielle et de la vision par ordinateur, les systèmes d'aide à la conduite avancés (ADAS) occupent aujourd'hui une place essentielle dans le développement des véhicules intelligents. Ces systèmes reposent sur des algorithmes de détection et d'analyse de l'environnement routier capables de fonctionner en temps réel afin d'améliorer la sécurité et l'assistance à la conduite.

Cependant, l'implémentation de ces algorithmes sur des plateformes embarquées pose plusieurs défis. Les modèles de détection d'objets basés sur l'apprentissage profond offrent généralement une grande précision, mais ils nécessitent également des ressources de calcul importantes. Cette contrainte devient particulièrement critique dans les systèmes embarqués où les capacités de calcul, la mémoire et la consommation énergétique sont limitées. Dans ce contexte, il est nécessaire d'optimiser l'exécution de ces modèles afin de garantir un traitement rapide tout en maintenant un niveau de précision satisfaisant. L'utilisation de différentes architectures matérielles telles que les GPU et les FPGA représente une solution potentielle pour améliorer les performances des systèmes embarqués.

Ainsi, la problématique principale de ce projet consiste à étudier et analyser l'implémentation de modèles de vision par ordinateur pour les applications ADAS sur différentes plateformes matérielles, notamment les GPU et les FPGA, afin d'évaluer leurs performances en termes de vitesse d'exécution, de précision et d'efficacité dans un environnement embarqué.

1.3 Étude et critique de l'existant

Les systèmes d'aide à la conduite avancés connaissent une évolution importante grâce aux progrès réalisés dans les domaines de la vision par ordinateur, de l'intelligence artificielle et des systèmes embarqués. Plusieurs grandes entreprises technologiques et constructeurs automobiles développent aujourd'hui des solutions ADAS intégrant des fonctionnalités avancées telles que la détection des véhicules, des piétons, des panneaux de signalisation, l'assistance au maintien de voie et le freinage automatique d'urgence.

Parmi les solutions existantes, le système *Tesla Autopilot* développé par Tesla repose sur des algorithmes de vision par ordinateur et d'apprentissage profond afin d'analyser l'environnement routier en temps réel à l'aide de caméras et de capteurs intelligents. Cette solution permet notamment la détection des obstacles, le maintien dans la voie et l'assistance à la conduite semi-autonome. De

son côté, la plateforme *Mobileye*, largement utilisée par plusieurs constructeurs automobiles tels que BMW, Audi et Nissan, propose des systèmes ADAS basés sur des caméras intelligentes capables de reconnaître les piétons, les véhicules et les panneaux de signalisation. Mobileye met également l'accent sur les technologies de prévention des collisions et l'analyse avancée de l'environnement routier.

L'entreprise NVIDIA propose également la plateforme *NVIDIA DRIVE*, une solution dédiée aux véhicules intelligents et autonomes. Cette plateforme combine des modèles d'intelligence artificielle avec des technologies de calcul haute performance afin d'assurer le traitement en temps réel des données issues des capteurs du véhicule. D'autres entreprises comme Bosch, Continental et Valeo développent également des systèmes ADAS intégrant des technologies de détection et de surveillance avancées afin d'améliorer la sécurité routière et de réduire les risques d'accidents.

Sur le plan logiciel, plusieurs modèles de détection d'objets sont utilisés dans les systèmes ADAS modernes. Les architectures de la famille YOLO (*You Only Look Once*) sont particulièrement appréciées pour leur rapidité d'exécution et leur capacité à fonctionner en temps réel. Les modèles SSD (*Single Shot Detector*) offrent également un bon compromis entre vitesse et précision, tandis que Faster R-CNN permet d'obtenir une meilleure précision au prix d'un temps de calcul plus élevé.

L'analyse de ces différentes solutions permet d'établir une comparaison structurée selon des critères pertinents pour les applications ADAS embarquées, comme présenté dans le tableau 1.1.

Système	Entreprise	Approche	Plateforme	Précision	Contraintes embarquées	Limite principale
Tesla Autopilot	Tesla	Vision pure (caméras)	GPU custom embarqué	Élevée	Propriétaire, fermé	Sensible aux conditions météo
Mobileye EyeQ	Mobileye/Intel	Caméra monoculaire	SoC dédié EyeQ	Très élevée	Faible consommation	Coût élevé
NVIDIA DRIVE	NVIDIA	Multi-capteurs + DNN	Orin/Xavier GPU	Très élevée	10–65 W	Consommation énergétique élevée
YOLO embarqué	Académique	Vision + DNN léger	Jetson / FPGA	Moyenne Élevée	< 10 W	mAP limité sur modèles compacts

Tableau 1.1 : Comparaison des solutions ADAS existantes

L'analyse de ces solutions met en évidence un compromis fondamental entre précision de

détection et efficacité sur plateforme embarquée. Les solutions commerciales telles que Mobileye et NVIDIA DRIVE offrent d'excellentes performances mais restent coûteuses et peu accessibles pour le prototypage académique. Les approches basées sur YOLO, bien que moins précises en valeur absolue, présentent un potentiel réel pour le déploiement embarqué sous contraintes strictes de mémoire et de consommation énergétique.

1.4 Cahier des charges

Dans le cadre de ce projet de fin d'études, l'objectif principal est de concevoir et de développer une solution intelligente de détection d'objets destinée aux systèmes avancés d'aide à la conduite (ADAS). Cette solution doit garantir un fonctionnement fiable, rapide et optimisé afin de répondre aux contraintes des systèmes embarqués utilisés dans le domaine automobile.

Le système proposé doit permettre la détection et l'analyse en temps réel des différents éléments présents dans l'environnement routier afin d'améliorer la sécurité de conduite et de faciliter l'intégration dans les futures plateformes de véhicules intelligents et autonomes.

Ainsi, la solution développée doit assurer :

- **La détection d'objets routiers en temps réel :** Le système doit être capable de détecter et localiser plusieurs catégories d'objets présents dans une scène routière, notamment les voitures, les camions, les motos, les piétons, les cyclistes, les panneaux de signalisation ainsi que les feux tricolores. Cette détection doit être réalisée sur des flux vidéo en temps réel afin de garantir une réactivité adaptée aux applications ADAS.
- **L'utilisation d'un modèle de deep learning optimisé :** La solution doit s'appuyer sur l'architecture YOLO pour assurer un compromis efficace entre précision de détection et rapidité d'exécution. Le modèle développé doit être capable d'offrir de bonnes performances tout en restant compatible avec les contraintes matérielles des plateformes embarquées.
- **L'optimisation du modèle par distillation de connaissances :** Afin de réduire la complexité du réseau de neurones tout en conservant une précision élevée, une approche de distillation de connaissances doit être appliquée. Cette méthode consiste à transférer les connaissances d'un modèle teacher complexe vers un modèle student plus léger et plus adapté aux systèmes embarqués.
- **Le traitement temps réel sur plateformes embarquées :** La solution doit assurer une vitesse d'inférence suffisante pour les applications de conduite assistée, avec un

objectif minimal de 25 images par seconde sur la plateforme NVIDIA Jetson Nano. Le système doit également maintenir une faible latence afin de garantir une réponse rapide aux événements détectés.

- **Le déploiement sur plusieurs plateformes matérielles :** Le modèle développé doit être compatible avec différentes plateformes de calcul :
 - un PC équipé d'un GPU NVIDIA pour l'entraînement et les tests
 - une carte NVIDIA Jetson Nano pour l'exécution embarquée
 - une carte FPGA ZedBoard Zynq-7020 pour l'accélération matérielle et l'évaluation des performances FPGA.
- **L'optimisation FPGA et la quantification du modèle :** Dans le cadre du déploiement FPGA, le système doit intégrer des techniques d'optimisation telles que la quantification INT8 via l'environnement Vitis AI afin de réduire l'utilisation des ressources matérielles et d'améliorer les performances d'inférence.
- **L'évaluation et la comparaison des performances :** La solution doit permettre une analyse comparative des différentes plateformes en utilisant plusieurs métriques de performance telles que :
 - le mAP (Mean Average Precision)
 - le nombre d'images par seconde (FPS)
 - la latence d'inférence
 - la consommation énergétique.
- **La visualisation des résultats de détection :** Une interface de démonstration doit permettre l'affichage des objets détectés en temps réel sur des images ou des flux vidéo, avec annotation des classes détectées et des boîtes englobantes (*bounding boxes*).

1.5 Choix méthodologique

Afin d'atteindre efficacement les objectifs de notre projet, nous avons opté pour la méthodologie **PDCA** (Plan, Do, Check, Act), qui constitue une approche itérative basée sur l'amélioration continue.

Le cycle PDCA repose sur quatre étapes principales permettant de structurer et d'optimiser le processus de développement, comme illustré dans la figure 1.2.



Figure 1.2 : Cycle PDCA

1.5.1 Application de la méthodologie PDCA

La méthodologie PDCA a servi de cadre directeur tout au long de la réalisation de ce projet, en assurant une progression claire et structurée. Elle a été appliquée selon les étapes suivantes :

- **Plan** : Cette première étape a consisté à identifier la problématique principale liée à l'implémentation de systèmes ADAS basés sur la vision par ordinateur. Nous avons défini les objectifs du projet, réalisé une étude de l'existant et sélectionné les outils et technologies appropriés, notamment les modèles de détection (YOLO), ainsi que les plateformes matérielles (GPU et FPGA). Un plan d'action a été établi, incluant la préparation des données, le choix des modèles et la définition de l'architecture du système.
- **Do** : Dans cette phase, nous avons procédé à l'implémentation du système. Cela inclut l'entraînement et l'optimisation des modèles de détection d'objets, leur déploiement sur une plateforme embarquée (Jetson Nano) ainsi que l'implémentation sur FPGA. Nous avons également développé les modules nécessaires au traitement des flux vidéo en temps réel.
- **Check** : Cette étape a été dédiée à l'évaluation des performances du système. Nous avons effectué des tests afin de mesurer la précision des modèles, le temps d'exécution (latence) et l'utilisation des ressources matérielles. Ces analyses ont permis d'identifier les limites et les axes d'amélioration.
- **Act** : Enfin, sur la base des résultats obtenus, nous avons apporté des améliorations au système. Ainsi que l'ajustement des paramètres afin d'obtenir un meilleur compromis entre précision et performance sur les plateformes embarquées.

1.5.2 Diagramme de Gantt

Le diagramme de Gantt présenté dans la figure 1.3 illustre la planification et l'évolution des différentes tâches réalisées tout au long du projet. Il couvre une période allant de février à juillet et met en évidence les principales phases du travail.

Chaque tâche est représentée par une barre horizontale indiquant sa durée et sa période d'exécution. Le diagramme commence par l'étude des besoins et la recherche bibliographique, suivies par le développement du pipeline sur PC. Ensuite, les phases d'implémentation sur GPU (Jetson Nano) et FPGA (ZedBoard) ont été réalisées, avant d'aboutir à une étape de comparaison des performances. Enfin, la rédaction du rapport s'est déroulée en parallèle durant toute la période du projet.

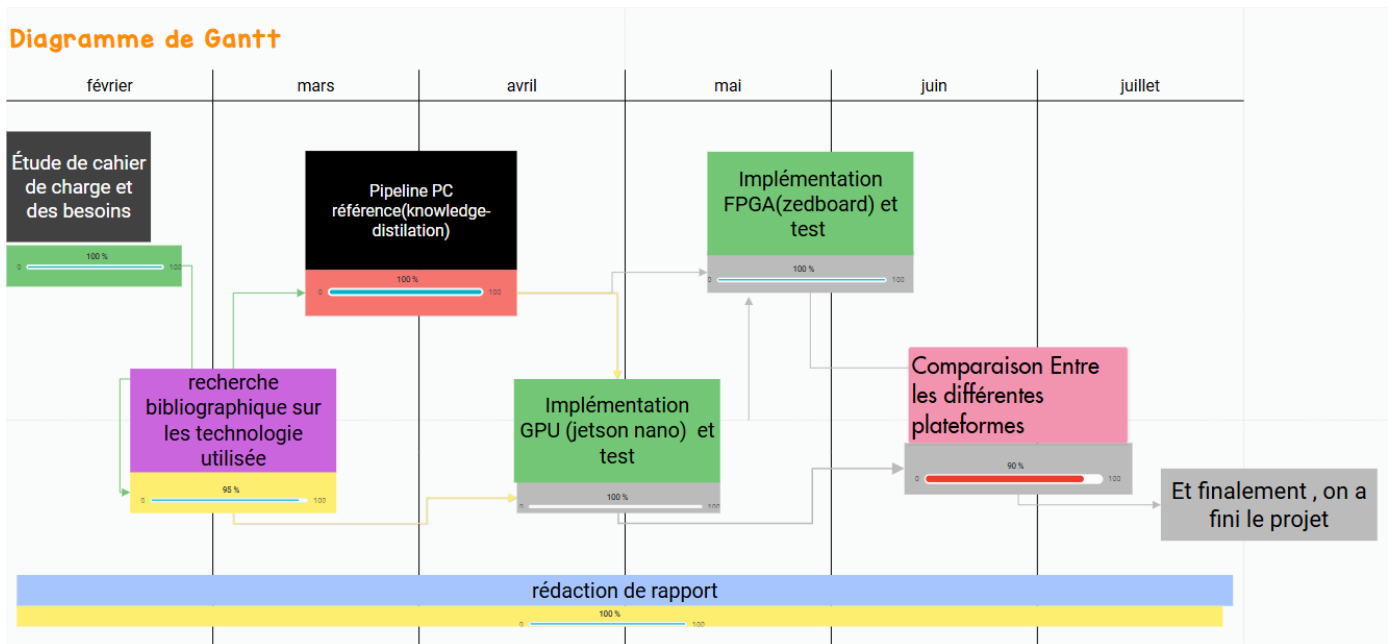


Figure 1.3 : diagramme de Gantt

Conclusion

Dans ce chapitre a permis de dresser un panorama complet du contexte du projet. Nous avons identifié les principales difficultés à surmonter et défini les objectifs à atteindre. Ces éléments constituent le point de départ de l'étude technique qui sera développée dans les chapitres suivants.

État de l'art

Plan

1	Systèmes ADAS : définition et architecture	11
2	Modèles de détection d'objets	14
3	Techniques d'optimisation	18
4	Distillation de connaissances	19
5	Datasets de référence pour la détection ADAS	22
6	Synthèse et justification des choix technologiques	24

Introduction

La détection d'objets en temps réel constitue une brique fondamentale des systèmes ADAS modernes. Ce chapitre dresse un état de l'art des technologies nécessaires à la conception du système proposé. Nous présentons successivement les architectures ADAS et leurs capteurs, les modèles de détection d'objets avec un focus sur la famille YOLO, les techniques d'optimisation embarquée, puis la distillation de connaissances. Le chapitre se conclut par une justification des choix technologiques retenus.

2.1 Systèmes ADAS : définition et architecture

Dans cette partie, nous allons présenter en détail le fonctionnement des systèmes avancés d'aide à la conduite (*ADAS*), en expliquant leurs principes de fonctionnement, les différentes technologies utilisées ainsi que les principaux capteurs intégrés dans ces systèmes, la détection des objets et l'assistance intelligente à la conduite.

2.1.1 Définition et enjeux

Un système ADAS désigne un ensemble de technologies électroniques intelligentes intégrées au véhicule afin d'assister le conducteur durant la conduite et d'améliorer la sécurité routière, le confort ainsi que l'efficacité de déplacement [1]. Ces systèmes reposent sur une combinaison de capteurs, d'algorithmes de traitement de données et de techniques d'intelligence artificielle permettant au véhicule de percevoir son environnement en temps réel, d'analyser les situations de conduite et de réagir de manière adaptée face aux différents scénarios routiers.

Les systèmes ADAS interviennent dans plusieurs fonctions essentielles telles que la détection d'obstacles, le freinage d'urgence automatique, l'assistance au maintien de voie, la reconnaissance des panneaux de signalisation, le régulateur de vitesse adaptatif ou encore la surveillance des angles morts. Pour assurer ces fonctionnalités, les véhicules modernes intègrent différents types de capteurs comme les caméras, les radars, les lidars et les capteurs ultrasoniques, chacun jouant un rôle spécifique dans la perception de l'environnement routier.

Selon la classification définie par la SAE International (*Society of Automotive Engineers*), le niveau d'automatisation des véhicules est réparti sur une échelle allant du niveau 0, correspondant à une conduite totalement manuelle sans assistance automatisée, jusqu'au niveau 5, représentant

une autonomie complète sans intervention humaine. Les systèmes ADAS actuellement déployés dans l'industrie automobile se situent principalement entre les niveaux 1 et 3, où certaines tâches de conduite peuvent être automatisées tout en nécessitant une supervision du conducteur [1], [2], comme illustré dans la figure 2.1.

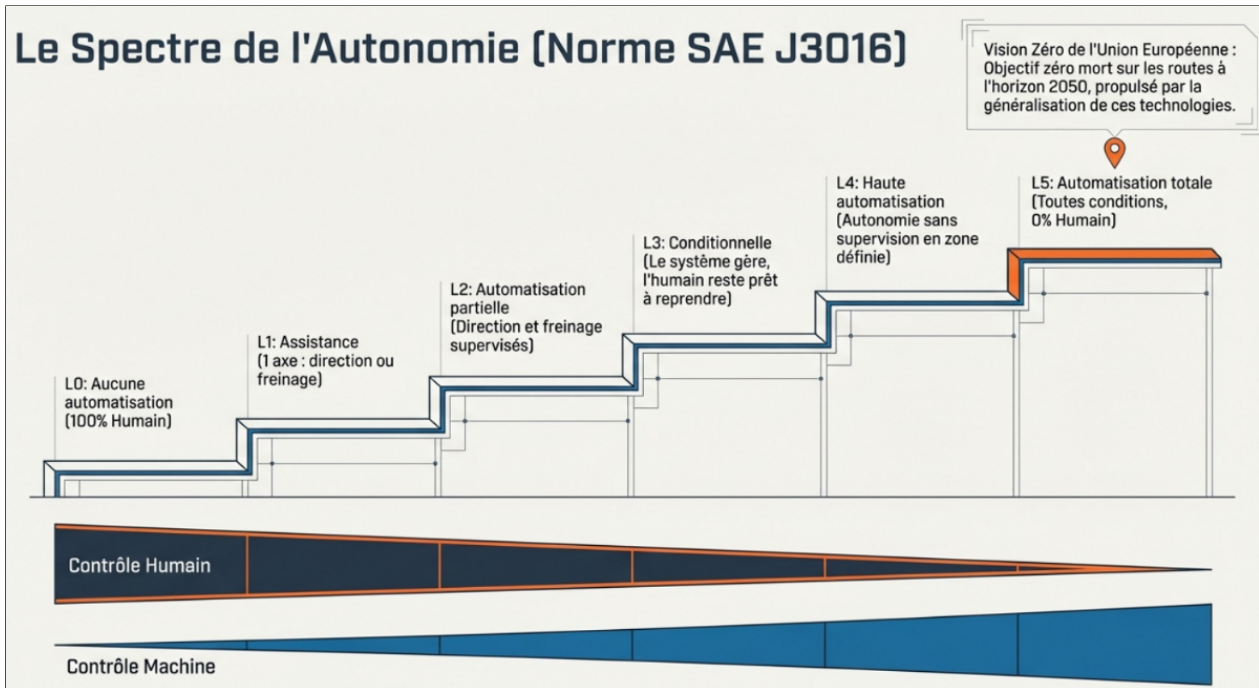


Figure 2.1 : Niveaux d'automatisation SAE

2.1.2 Architecture sensorielle

Les systèmes ADAS s'appuient sur une fusion multimodale de capteurs complémentaires permettant une perception robuste de l'environnement [2] :

- **Caméras monoculaires et stéréoscopiques** : principal vecteur d'information visuelle, permettant la détection de voies, d'obstacles, de piétons et de panneaux de signalisation. Leur faible coût et leur richesse en information en font le capteur dominant dans les ADAS.
- **Radars (FMCW)** : mesure de la distance et de la vitesse relative des obstacles par émission d'ondes électromagnétiques. Robustes aux conditions météorologiques défavorables (pluie, brouillard, nuit).
- **LiDAR (*Light Detection and Ranging*)** : génération d'un nuage de points 3D haute résolution par émission d'impulsions laser. Offre une cartographie précise de l'environnement mais reste coûteux.

- **Capteurs ultrasoniques** : utilisés pour la détection de proximité à courte distance, notamment en manœuvres de stationnement.
- **GPS/IMU** : localisation globale et mesure des données inertielles pour la fusion de capteurs.

2.1.3 Fonctions principales

Les fonctionnalités ADAS se déclinent en plusieurs catégories fonctionnelles :

- **Détection et classification d'obstacles** (*Object Detection*) : identification en temps réel des véhicules, piétons, cyclistes et obstacles statiques.
- **Maintien dans la voie** (*Lane Keeping Assist LKA*) : détection des marquages au sol et correction automatique de la trajectoire.
- **Régulateur de vitesse adaptatif** (*Adaptive Cruise Control ACC*) : maintien d'une distance de sécurité avec le véhicule précédent.
- **Freinage automatique d'urgence** (*Autonomous Emergency Braking AEB*) : déclenchement automatique du freinage en cas de risque de collision imminent.
- **Détection de la somnolence** (*Driver Monitoring System DMS*) : surveillance de l'état d'attention du conducteur par analyse du visage.
- **Vision nocturne** (*Night Vision*) : amélioration de la visibilité par caméra infrarouge.

comme illustré dans la figure 2.2.



Figure 2.2 : Exemple adas VALEO[3]

2.1.4 Pipeline de traitement d'image

La vision par ordinateur est la discipline qui vise à doter les machines d'une capacité à interpréter et comprendre le contenu des images numériques [4]. Appliquée aux ADAS, elle se concentre principalement sur la détection d'objets, la segmentation sémantique et le suivi de cibles mobiles.

Le pipeline classique de traitement en vision par ordinateur comprend :

1. **Acquisition de l'image** : capture d'une trame vidéo par la caméra embarquée.
2. **Prétraitement** : redimensionnement, normalisation des valeurs de pixels, augmentation de données.
3. **Extraction de caractéristiques** : identification des représentations pertinentes via un réseau de neurones profond.
4. **Détection / classification** : localisation et identification des objets présents dans la scène.
5. **Post-traitement** : suppression des doublons par NMS (*Non-Maximum Suppression*), filtrage des détections.

comme illustré dans la figure 2.3.

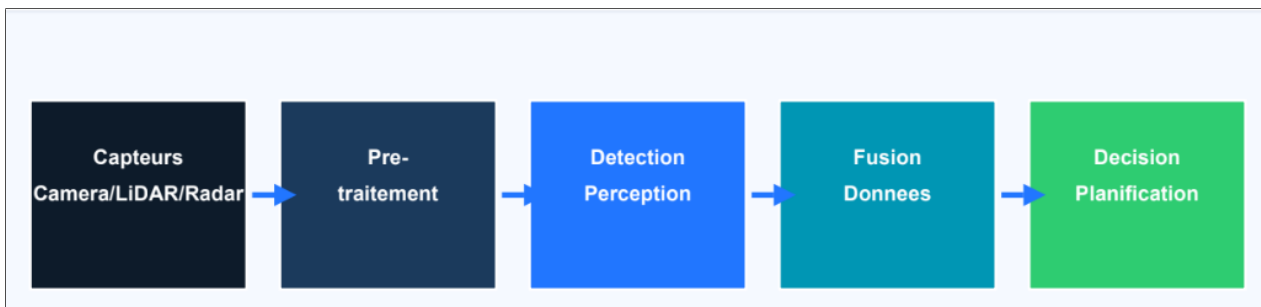


Figure 2.3 : Pipeline de traitement ADAS

2.2 Modèles de détection d'objets

La détection d'objets (Object Detection) est une branche importante de la vision par ordinateur qui consiste à identifier automatiquement la présence d'objets dans une image ou une vidéo, tout en déterminant leur position exacte à l'aide de boîtes englobantes

2.2.1 Détecteurs à deux étapes (Two-Stage Detectors)

Les détecteurs à deux étapes réalisent la détection en deux phases successives :

1. Génération des régions d'intérêt (*Region Proposals*)
2. Classification et raffinement des objets détectés.

Cette approche offre généralement une très grande précision, mais nécessite un temps de calcul plus important.

Les modèles les plus connus de cette famille sont :

- R-CNN
- Fast R-CNN
- Faster R-CNN
- Mask R-CNN

Le modèle Faster R-CNN est considéré comme une référence en termes de précision, notamment dans les applications nécessitant une détection fine des objets. Cependant, sa complexité de calcul le rend moins adapté aux applications temps réel embarquées comme les systèmes ADAS.

2.2.2 Détecteurs à une étape (One-Stage Detectors)

Les détecteurs à une étape réalisent directement la localisation et la classification des objets en une seule passe du réseau neuronal. Cette approche réduit fortement le temps d'inférence et permet d'atteindre des performances temps réel.

Les principaux modèles de cette catégorie sont :

- SSD (*Single Shot Detector*)
- RetinaNet
- YOLO (*You Only Look Once*)

Ces architectures sont particulièrement adaptées aux systèmes embarqués et aux applications nécessitant une faible latence, comme :

- la conduite autonome
- la vidéosurveillance temps réel
- les drones
- la robotique mobile

Parmi ces modèles, la famille YOLO s'est imposée comme l'une des solutions les plus performantes grâce à son excellent compromis entre rapidité et précision.

2.2.3 La famille YOLO

YOLO (*You Only Look Once*) est une famille de modèles de détection d'objets en temps réel, introduite par Redmon et al. en 2016 [5]. Son principe fondateur consiste à reformuler la détection comme un unique problème de régression, traité en une seule passe du réseau.

2.2.3.1 Principe de fonctionnement

L'image d'entrée est divisée en une grille de SS cellules. Chaque cellule prédit B boîtes englobantes, leurs scores de confiance et les probabilités de classes. La prédiction pour chaque boîte comprend le vecteur $(x, y, w, h, C, p_1, \dots, p_K)$ où :

- (x, y) : coordonnées du centre de la boîte relatives à la cellule,
- (w, h) : dimensions de la boîte relatives à l'image,
- C : score de confiance de la détection,
- p_k : probabilité conditionnelle de la classe k .

2.2.3.2 Fonction de perte YOLO

La fonction de perte multi-composantes de YOLO est définie par :

$$\mathcal{L} = \underbrace{\lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{ij}^{\text{obj}} [(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2]}_{\text{Localisation}} + \underbrace{\lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{ij}^{\text{obj}} [(\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2]}_{\text{Taille}} \quad (2.1)$$

$$+ \underbrace{\sum_{i=0}^{S^2} \sum_{j=0}^B 1_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2}_{\text{Confiance}} + \underbrace{\sum_{i=0}^{S^2} 1_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2}_{\text{Classification}} \quad (2.2)$$

où 1_{ij}^{obj} vaut 1 si un objet est présent dans la cellule i , $\lambda_{\text{coord}} = 5$ et $\lambda_{\text{noobj}} = 0.5$ sont des hyperparamètres de pondération.

2.2.3.3 Évolution de la famille YOLO

Le tableau 2.1 résume l'évolution des versions YOLO et leurs caractéristiques principales :

Version	Année	mAP (COCO)	FPS (GPU)*	Apport principal
YOLOv1	2016	63.4	45	Détection en une passe
YOLOv3	2018	55.3	20	Détection multi-échelle
YOLOv5s	2020	56.8	140	Auto-ancres, CSP backbone
YOLOv7	2022	56.8	161	E-ELAN, GELAN
YOLOv8n	2023	52.9	500+	Anchor-free, tâches multiples
YOLOv9c	2024	53.0	300+	PGI, GELAN amélioré
YOLOv11n	2024	39.5	700+	Architecture légère optimisée

Tableau 2.1 : Évolution des versions YOLO et performances sur COCO val2017

* Les valeurs FPS sont mesurées sur GPU NVIDIA haute performance (RTX 3080 ou équivalent) à résolution d'entrée 640640. Les valeurs mAP sont rapportées sur COCO val2017 dans les conditions expérimentales définies par chaque publication originale.

2.2.4 Métriques d'évaluation

Les performances d'un détecteur sont évaluées via les métriques suivantes :

- **IoU** (*Intersection over Union*) : ratio entre l'intersection et l'union de la boîte prédite et de la boîte réelle.

$$\text{IoU} = \frac{|\hat{B} \cap B^*|}{|\hat{B} \cup B^*|} \quad (2.3)$$

- **Précision / Rappel** : précision = $\text{TP}/(\text{TP}+\text{FP})$, rappel = $\text{TP}/(\text{TP}+\text{FN})$.
- **AP** (*Average Precision*) : aire sous la courbe précision-rappel pour une classe donnée.
- **mAP** (*mean Average Precision*) : moyenne des AP sur toutes les classes. La notation mAP@0.5 correspond à un seuil IoU de 0.5.
- **FPS** (*Frames Per Second*) : débit de traitement, critique pour le temps réel. Un minimum de 25 FPS est généralement requis pour les applications ADAS.
- **Latence d'inférence** : temps de traitement d'une trame, exprimé en millisecondes.

2.3 Techniques d'optimisation

Le déploiement des modèles d'intelligence artificielle dans les systèmes ADAS embarqués nécessite l'utilisation de différentes techniques d'optimisation afin de réduire la complexité des réseaux de neurones tout en maintenant des performances satisfaisantes. En effet, les plateformes embarquées disposent généralement de ressources limitées en termes de mémoire, de puissance de calcul et de consommation énergétique. Ainsi, plusieurs approches ont été développées pour améliorer l'efficacité des modèles de vision par ordinateur.

• **Quantification** : La quantification consiste à réduire la précision numérique des poids et des activations du réseau de neurones. Les modèles sont généralement entraînés en précision flottante 32 bits (*FP32*), mais cette représentation nécessite une importante capacité mémoire et un temps de calcul élevé. L'utilisation de représentations plus légères telles que INT8, INT4 ou même binaire permet de réduire considérablement la taille du modèle ainsi que la consommation mémoire. Cette technique offre également une accélération significative de l'inférence sur les plateformes embarquées. Par exemple, le passage de FP32 vers INT8 permet souvent une réduction mémoire d'environ 4 fois tout en conservant une précision acceptable pour les applications ADAS.

• **Élagage (*Pruning*)** : L'élagage consiste à supprimer les poids, neurones ou filtres jugés peu importants dans le réseau de neurones. En effet, après l'entraînement, plusieurs connexions contribuent faiblement au résultat final du modèle. Cette technique permet de réduire le nombre d'opérations nécessaires lors de l'inférence et d'alléger la taille du modèle. On distingue principalement :

- l'élagage non structuré, qui supprime des poids individuels ;
- l'élagage structuré, qui élimine des filtres ou des couches complètes afin de faciliter l'accélération matérielle.

L'élagage améliore ainsi les performances d'exécution tout en réduisant la consommation énergétique des systèmes embarqués.

• **Distillation de connaissances (*Knowledge Distillation*)** : La distillation de connaissances est une technique permettant de transférer les connaissances d'un modèle complexe et précis appelé *Teacher* vers un modèle plus léger appelé *Student*. Le modèle *Teacher*, généralement volumineux, produit des prédictions riches en informations qui servent à guider l'apprentissage du modèle *Student*. Cette approche permet d'obtenir des modèles compacts capables d'atteindre des performances proches du modèle initial tout en réduisant fortement les besoins en calcul et en mémoire. en détaillée en section 2.4.

• **Compilation et optimisation du graphe de calcul** : Plusieurs frameworks spécialisés permettent d’optimiser automatiquement l’exécution des modèles d’intelligence artificielle sur des architectures matérielles spécifiques. Parmi les outils les plus utilisés :

- **TensorRT** pour les GPU NVIDIA
- **Vitis AI** pour les FPGA Xilinx
- **OpenVINO** pour les processeurs Intel.

Ces outils réalisent différentes optimisations telles que :

- la fusion des couches (*Layer Fusion*) ;
- la réduction des accès mémoire ;
- l’optimisation des opérations parallèles ;
- la génération de graphes adaptés au matériel cible.

Ces optimisations permettent d’améliorer significativement le temps d’exécution et l’efficacité énergétique des modèles.

• **Utilisation d’opérateurs légers** : Une autre approche consiste à modifier directement l’architecture du réseau afin de réduire sa complexité computationnelle. Les convolutions standards, très coûteuses en calcul, peuvent être remplacées par des convolutions *Depthwise Separable*, utilisées notamment dans les architectures MobileNet.

Cette technique décompose l’opération de convolution en deux étapes plus simples :

- une convolution spatiale appliquée indépendamment sur chaque canal ;
- une convolution ponctuelle (*Pointwise Convolution*) permettant de combiner les informations.

Cela permet de réduire fortement le nombre d’opérations mathématiques ainsi que la taille du modèle, tout en conservant des performances satisfaisantes pour les applications temps réel.

L’association de ces différentes techniques d’optimisation permet de concevoir des modèles de vision par ordinateur plus légers, plus rapides et mieux adaptés aux contraintes des systèmes ADAS embarqués. Ces approches jouent un rôle essentiel dans le déploiement efficace des applications d’intelligence artificielle sur des plateformes à ressources limitées.

2.4 Distillation de connaissances

Dans cette section, nous expliquerons en détail la technique de distillation des connaissances.

2.4.1 Motivations et principe

Le déploiement de modèles de détection sur des plateformes embarquées à ressources limitées soulève un défi fondamental : concilier précision et efficacité. La distillation de connaissances (*Knowledge Distillation* *KD*), introduite par Hinton et al. [6], propose une solution élégante à ce problème en transférant les connaissances acquises par un modèle complexe (*teacher*) vers un modèle léger (*student*).

L'intuition sous-jacente est que les distributions de probabilité produites par le modèle enseignant sur les classes contiennent une information structurelle riche les *soft labels* bien au-delà des simples étiquettes dures (*hard labels*) de la supervision classique, comme illustré dans la figure 2.4.

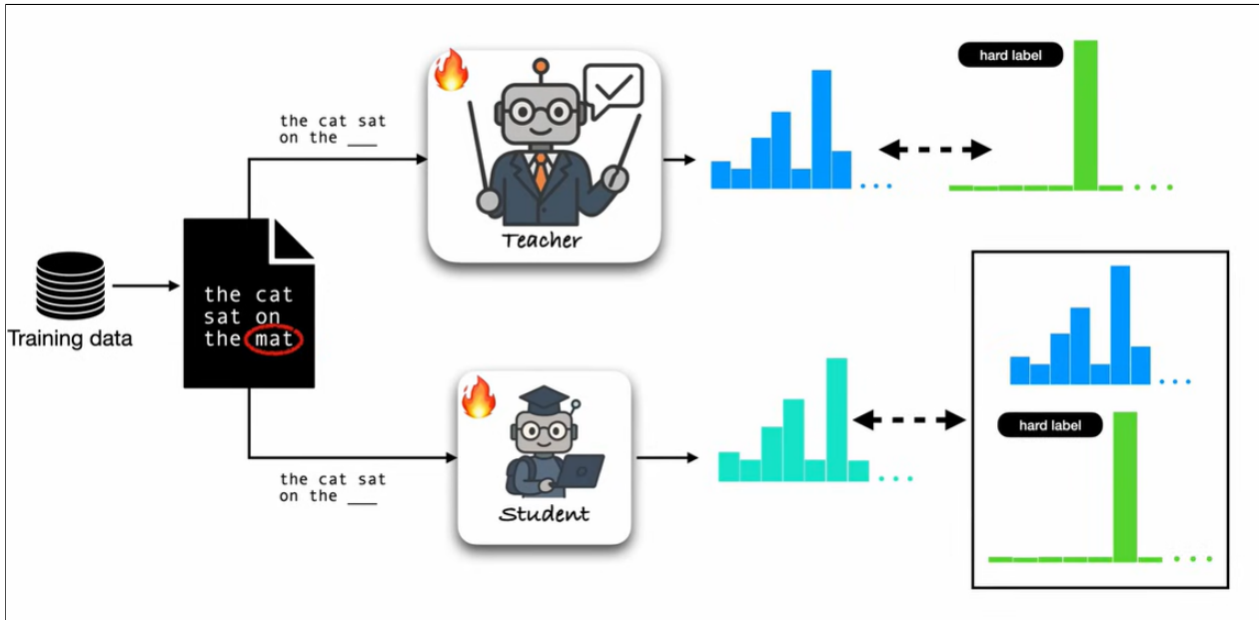


Figure 2.4 : exemple knowledge distillation

2.4.2 Formulation mathématique

Pour bien comprendre l'utilisation de la distillation, les formules mathématiques sont essentielles.

2.4.2.1 Températures de softmax

Les logits du modèle enseignant sont adoucis via un paramètre de température T :

$$p_i^T = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \quad (2.4)$$

Pour $T = 1$, on retrouve le softmax standard. Pour $T > 1$, la distribution devient plus douce,

révélaient les similarités inter-classes apprises par le modèle enseignant.

2.4.2.2 Fonction de perte de distillation

La fonction de perte globale du modèle étudiant est une combinaison pondérée :

$$\mathcal{L}_{\text{total}} = \alpha \cdot \mathcal{L}_{\text{CE}}(y, \hat{p}_S) + (1 - \alpha) \cdot T^2 \cdot D_{\text{KL}}(p_{\text{teacher}}^T \parallel p_{\text{student}}^T) \quad (2.5)$$

où :

- $\mathcal{L}_{\text{CE}}$: entropie croisée avec les étiquettes réelles (*hard labels*),
- D_{KL} : divergence de Kullback-Leibler entre les distributions adoucies,
- $\alpha \in [0, 1]$: coefficient d'équilibrage,
- T^2 : facteur de compensation de l'échelle des gradients.

2.4.3 Taxonomie des méthodes de distillation

Il existe plusieurs types de méthodes de distillation, chacune étant utilisée dans des conditions spécifiques.

2.4.3.1 Distillation des logits (Response-based KD)

La méthode originale d'Hinton [6] consiste à aligner les sorties finales du *student* sur celles du *teacher*. Simple à mettre en uvre, elle ne transfère que la connaissance de la couche de sortie.

2.4.3.2 Distillation des caractéristiques intermédiaires (Feature-based KD)

FitNets [7] propose de superviser les représentations intermédiaires du *student* pour qu'elles imitent celles du *teacher*. La perte associée est :

$$\mathcal{L}_{\text{feat}} = \|\phi(F_S) - F_T\|_2^2 \quad (2.6)$$

où F_S et F_T sont les cartes de caractéristiques du *student* et du *teacher*, et ϕ est une transformation linéaire permettant d'aligner leurs dimensions.

2.4.3.3 Distillation par cartes d'attention (Attention Transfer)

Zagoruyko et al. [8] proposent de transférer les cartes d'attention du *teacher* au *student*, ces cartes encodant les régions de l'image jugées discriminantes. La carte d'attention est calculée

comme :

$$A_l = \frac{F_l^2}{\|F_l^2\|_1} \quad (2.7)$$

2.4.3.4 Distillation relationnelle (Relational KD)

Park et al. [9] exploitent les relations mutuelles entre exemples d’un mini-lot, en transférant la structure relationnelle des représentations plutôt que les représentations elles-mêmes.

2.4.3.5 Distillation auto-supervisée (Self-distillation)

Une variante ne requérant pas de *teacher* externe : le modèle se distille lui-même entre ses propres couches ou entre différentes passes d’entraînement.

2.4.4 Distillation pour la détection d’objets

L’application de la distillation aux détecteurs comme YOLO présente des défis spécifiques liés à la nature dense des prédictions. Plusieurs approches ont été proposées [10] :

- **MGD** (*Masked Generative Distillation*) : transfert des caractéristiques FPN masquées aléatoirement, forçant le *student* à reconstruire les régions manquantes.
- **CWD** (*Channel-Wise Distillation*) : distillation canal par canal des cartes de caractéristiques.
- **Logit Mimicking** : adaptation de la distillation des logits aux sorties denses des détecteurs.

2.5 Datasets de référence pour la détection ADAS

Les systèmes ADAS reposent sur des modèles entraînés sur des données annotées représentatives des conditions réelles de conduite. Plusieurs datasets publics ont été développés pour répondre à ces besoins, chacun présentant des caractéristiques, des avantages et des limites spécifiques [2].

2.5.1 KITTI

Le dataset KITTI [2] est l’un des benchmarks les plus utilisés dans le domaine de la perception autonome. Il contient des séquences vidéo capturées depuis un véhicule instrumenté en milieu urbain et périurbain allemand, couvrant la détection de véhicules, de piétons et de cyclistes en conditions réelles. Ses annotations incluent des boîtes englobantes 2D et 3D, ce qui le rend particulièrement adapté aux approches multi-capteurs.

Avantages : données réelles de conduite, annotations 3D précises, scènes variées jour/nuit.

Limites : faible couverture des panneaux de signalisation, pas de feux de circulation annotés, taille limitée.

2.5.2 BDD100K

BDD100K [2] est un large dataset de conduite autonome contenant 100 000 séquences vidéo capturées dans des conditions très variées : différentes villes américaines, conditions météorologiques (pluie, neige, brouillard), plages horaires (jour, nuit, crépuscule). Il couvre un large éventail de classes : véhicules, piétons, cyclistes, feux de signalisation et panneaux routiers.

Avantages : très grande diversité visuelle, annotations multi-classes incluant feux et panneaux, adapté aux conditions réelles de conduite.

Limites : annotations de panneaux peu détaillées par catégorie, forte dominance de scènes américaines, annotations parfois bruitées.

2.5.3 COCO (*Common Objects in Context*)

COCO [2] est un dataset généraliste de référence pour la détection d'objets. Il contient plus de 330 000 images annotées couvrant 80 classes d'objets courants, parmi lesquelles plusieurs sont directement pertinentes pour les applications ADAS : personnes, voitures, camions, bus, motos, bicyclettes et feux de signalisation.

Avantages : très large volume de données, grande diversité de scènes, standard de référence pour l'évaluation des modèles (mAP COCO val2017), support natif dans Ultralytics YOLOv8 [11].

Limites : pas orienté conduite, absence de panneaux routiers annotés, contextes non routiers majoritaires.

2.5.4 GTSDb (*German Traffic Sign Detection Benchmark*)

GTSDb [2] est un dataset spécialisé dans la détection de panneaux de signalisation routière. Il contient des images annotées par boîtes englobantes couvrant 43 catégories de panneaux issus du code de la route européen, capturées dans des conditions variées d'éclairage et de distance depuis un véhicule en mouvement.

Avantages : annotations précises et fiables, couverture complète des panneaux européens, images en conditions réelles de conduite, format compatible YOLO [11].

Limites : ne couvre pas les objets ADAS généraux (piétons, véhicules, feux), taille modeste,

limité à la signalisation routière.

2.5.5 Europe Signs

Europe Signs est un dataset complémentaire disponible via la plateforme Roboflow, couvrant un large ensemble de panneaux de signalisation européens répartis en plus de 50 catégories. Il inclut des panneaux de limitation de vitesse, d’interdiction, de danger, de priorité et d’obligation capturés dans des conditions routières réelles variées.

Avantages : complémentaire à GTSDb sur le périmètre européen, annotations fournies nativement au format YOLO, téléchargement simplifié via l’API Roboflow.

Limites : ne couvre pas les objets ADAS généraux (piétons, véhicules, feux), qualité d’annotation variable selon les contributeurs.

2.6 Synthèse et justification des choix technologiques

À l’issue de l’analyse comparative menée dans ce chapitre, cette section justifie les choix technologiques retenus pour la conception du système proposé, en s’appuyant sur les critères propres aux applications ADAS embarquées : précision de détection, latence d’inférence, compatibilité matérielle et efficacité mémoire.

2.6.1 Choix du modèle de détection : YOLOv8

Parmi les architectures de détection étudiées, YOLOv8 [11] est retenu comme modèle de base pour les raisons suivantes.

Précision et vitesse. YOLOv8n atteint un mAP de 52,9 sur COCO val2017 avec un débit supérieur à 500 FPS sur GPU, bien au-delà du seuil minimal de 25 FPS requis pour les applications temps réel [2]. Les détecteurs two-stage tels que Faster R-CNN **faster_rcnn_ref**, bien que plus précis en valeur absolue, présentent une latence incompatible avec les contraintes embarquées.

Architecture anchor-free. La prédiction directe du centre de l’objet, introduite dans YOLOv8 [11], supprime la sensibilité aux hyperparamètres d’ancres et améliore la généralisation sur des scènes routières variées.

Tête découplée. La séparation explicite des branches de classification et de localisation [11] facilite l’application ultérieure de la distillation de connaissances, en permettant d’aligner indépendamment chaque composante de sortie entre le modèle enseignant et le modèle étudiant.

Déployabilité. L’écosystème Ultralytics offre une chaîne d’exportation native vers TensorRT, ONNX et OpenVINO [11], ce qui simplifie le déploiement sur plateformes embarquées telles que NVIDIA Jetson. YOLOv11, bien que plus récent, présente un mAP de 39,5 dans sa configuration légère [2], inférieur à YOLOv8n, ce qui le disqualifie pour notre cas d’usage.

Modèle	mAP COCO	FPS	Anchor-free	Export embarqué
Faster R-CNN [12]	42.7	7	Non	Difficile
SSD [13]	23.2	46	Non	Moyen
YOLOv5s [13]	37.4	140	Non	Bon
YOLOv8n [11]	52.9	500+	Oui	Très bon
YOLOv11n	39.5	700+	Oui	Bon

Tableau 2.2 : Comparaison des architectures de détection pour le contexte ADAS embarqué

2.6.2 Choix de la technique d’optimisation : distillation de connaissances

Face aux contraintes mémoire et calculatoires des plateformes embarquées, la distillation de connaissances [6] est retenue comme technique d’optimisation principale. Deux variantes ont été évaluées.

Distillation par les sorties (Response-based KD). La méthode originale d’Hinton [6] aligne les distributions de probabilité softmax du modèle étudiant sur celles du modèle enseignant via la divergence KL. Sa simplicité de mise en uvre en fait un bon point de départ, mais elle ne transfère que l’information de la couche de sortie, ce qui limite le gain de précision à 1–2 % de mAP.

Distillation par les caractéristiques intermédiaires (Feature-based KD). L’approche FitNets [7] supervise les représentations internes du modèle étudiant pour qu’elles imitent celles du modèle enseignant. Appliquée aux cartes FPN de YOLOv8, cette stratégie – notamment via MGD (*Masked Generative Distillation*) [10] – permet un gain de 3–5 % de mAP tout en conservant la légèreté du modèle étudiant. C’est cette approche qui est retenue dans notre travail, car elle transfère une connaissance structurelle plus riche sur les régions discriminantes de l’image, particulièrement pertinente pour la détection de petits objets en conditions routières.

Méthode	Type	Gain mAP	Complexité
KD Hinton [6]	Response-based	+1–2 %	Faible
FitNets [7]	Feature-based	+2–4 %	Moyenne
AT [8]	Attention-based	+1–3 %	Moyenne
MGD [10]	Feature-based	+3–5 %	Moyenne
RKD [9]	Relation-based	+2–4 %	Élevée
Choix retenu : MGD (Feature-based KD)			

Tableau 2.3 : Comparaison des méthodes de distillation choix retenu pour YOLOv8

2.6.3 Stratégie de quantification pour le déploiement embarqué

Afin d’optimiser davantage l’exécution du modèle étudiant sur la plateforme cible, une quantification post-entraînement est appliquée en complément de la distillation. Deux niveaux de précision sont considérés [2].

FP16 (*Half Precision*). La représentation en virgule flottante 16 bits réduit la taille du modèle et la consommation mémoire d’un facteur 2 par rapport au FP32, sans dégradation perceptible de la précision. Elle est nativement supportée par les GPU NVIDIA via les Tensor Cores et constitue le premier palier de quantification appliqué après la distillation.

INT8. La quantification entière 8 bits réduit la taille du modèle d’un facteur 4 par rapport au FP32 et accélère significativement l’inférence sur les accélérateurs embarqués tels que NVIDIA Jetson via TensorRT [11]. Une calibration sur un sous-ensemble représentatif du dataset est nécessaire pour préserver la précision. Le tableau ci-dessous résume l’impact attendu de quantification.

Précision	Taille modèle	Accélération	Perte mAP	Support
FP32 (référence)	1	1	–	Universel
FP16	0,5	1,5–2	<0,5 %	Tensor Cores
INT8	0,25	2–4	<1 %	TensorRT, Vitis AI

Tableau 2.4 : Impact de la quantification FP16 et INT8 sur YOLOv8 pour déploiement embarqué

La stratégie retenue est une distillation en FP32, suivie d’une exportation TensorRT en INT8

avec calibration, permettant d’atteindre le meilleur compromis précision/vitesse sur la plateforme cible.

2.6.4 Choix des datasets retenus : GTSDb et Europe Signs

L’analyse des datasets présentée en section 2.5 conduit à retenir la combinaison **GTSDb** et **Europe Signs** comme sources principales pour la construction du dataset d’entraînement, pour les raisons suivantes.

Ces deux datasets sont les seuls à fournir des annotations précises et détaillées par catégorie de panneaux de signalisation routière européens, couvrant les limitations de vitesse, les interdictions, les dangers et les obligations. Cette granularité est indispensable pour un système ADAS capable de reconnaître et d’interpréter la signalisation routière en conditions réelles.

Par ailleurs, les images de ces deux sources sont capturées depuis un véhicule en mouvement dans des environnements routiers réels, incluant des conditions variées d’éclairage, de distance et d’angle de vue. Cette diversité visuelle améliore la robustesse du modèle face aux variations rencontrées en conditions de conduite réelle [1].

Afin de pallier l’absence de classes ADAS générales (piétons, véhicules, feux de signalisation) dans ces deux datasets, une étape d’enrichissement par pseudo-labellisation est appliquée à l’aide d’un modèle YOLOv8x pré-entraîné sur COCO [11]. Cette approche permet d’obtenir un dataset final couvrant l’ensemble des objets nécessaires au système ADAS proposé, sans recourir à une annotation manuelle coûteuse.

2.6.5 Récapitulatif des choix

Le tableau 2.5 synthétise l’ensemble des technologies, architectures et données retenus pour la réalisation du projet, ainsi que les raisons ayant motivé ces choix en fonction des contraintes de performance, de précision et d’exécution temps réel.

Composant	Alternatives étudiées	Choix retenu
Détecteur de base	R-CNN, SSD, YOLOv5/v7/v11	YOLOv8n [11]
Technique d'optimisation	Pruning, quantification seule	Distillation KD [6]
Type de KD	Response, Attention, Relation	Feature-based MGD [10]
Quantification	FP32, FP16, INT8	FP16 + INT8 (TensorRT)
Dataset panneaux	KITTI, BDD100K, COCO	GTSDb + Europe Signs [2]

Tableau 2.5 : Récapitulatif des choix technologiques retenus

Conclusion

Ce chapitre a passé en revue les principales technologies intervenant dans la conception d'un système de détection embarqué pour les ADAS. L'analyse comparative des architectures a conduit à retenir YOLOv8 pour son compromis vitesse-précision et sa compatibilité avec les contraintes embarquées. La distillation de connaissances a été identifiée comme la technique d'optimisation la plus adaptée pour réduire la complexité du modèle sans perte significative de précision. Ces choix guident directement la conception présentée au chapitre suivant.

Conception et choix technologiques

Plan

1	Analyse des besoins	30
2	Architecture globale du système	32
3	Architectures partielles par étape	33
4	Choix technologiques	37

Introduction

Ce chapitre est consacré à la conception du système développé dans le cadre de ce projet. Nous présentons d’abord les besoins auxquels le système doit répondre, puis nous décrivons comment il a été organisé et structuré. Nous expliquons enfin les différents outils et composants retenus pour sa réalisation.

3.1 Analyse des besoins

L’analyse des besoins vise à définir précisément les fonctionnalités attendues du système ainsi que les contraintes de performance et de qualité à respecter, issues des orientations stratégiques de Capgemini Engineering et des spécifications propres aux applications ADAS embarquées .

3.1.1 Besoins fonctionnels

Les besoins fonctionnels définissent l’ensemble des services que le système doit fournir pour répondre aux objectifs de l’application ADAS. Sept fonctionnalités principales ont été identifiées :

- **F1 Détection multi-classes** : identifier et localiser simultanément plusieurs catégories d’objets (véhicules, piétons, cyclistes, feux de signalisation, panneaux routiers) dans un flux vidéo en temps réel.
- **F2 Distillation des connaissances** : entraîner un modèle *student* compact guidé par un modèle *teacher* performant, avec conservation d’au moins 90 % de la précision (mAP) du *teacher*.
- **F3 Traitement temps réel** : le pipeline complet (prétraitement, inférence, post-traitement) doit atteindre un débit minimal de 25 FPS sur la plateforme Jetson Nano et sur PC de référence, conformément aux exigences ADAS .
- **F4 Quantification du modèle** : appliquer les techniques de quantification FP16 et INT8 afin de réduire l’empreinte mémoire et d’accélérer l’inférence, tout en évaluant leur impact sur la précision du modèle.
- **F5 Déploiement sur Jetson Nano** : déployer et exécuter le modèle optimisé sur la plateforme embarquée Jetson Nano , avec évaluation des performances en temps réel, de la latence et de l’utilisation des ressources matérielles.
- **F6 Déploiement FPGA** : porter et optimiser le modèle sur la ZedBoard via le flux Vitis AI , avec quantification INT8 et évaluation des performances sur cette cible.

- **F7 Évaluation comparative** : comparer rigoureusement les trois plateformes (PC, Jetson Nano, ZedBoard) selon les métriques mAP, FPS, latence et consommation énergétique (W).

3.1.2 Besoins non fonctionnels

Les besoins non fonctionnels correspondent aux contraintes de qualité, de performance et de fiabilité auxquelles le système doit satisfaire :

- **Précision de détection** : le modèle doit maintenir un mAP@0.5 supérieur ou égal à 0,50 sur le dataset ADAS afin de garantir une perception fiable de l’environnement routier dans un contexte de sécurité critique .
- **Rapidité d’exécution** : le système doit traiter les images avec une latence inférieure ou égale à 40 ms par trame, soit un débit minimal de 25 FPS sur Jetson Nano, condition indispensable pour la réactivité des applications ADAS .
- **Faible consommation énergétique** : la consommation de la plateforme embarquée Jetson Nano ne doit pas dépasser 10 W afin de respecter les contraintes d’alimentation des systèmes embarqués .
- **Compacité du modèle** : le modèle *student* final doit présenter une taille inférieure ou égale à 20 Mo afin de s’adapter aux capacités mémoire limitées des plateformes embarquées ciblées .
- **Robustesse** : le système doit conserver des performances satisfaisantes dans différentes conditions de luminosité, météorologiques et de circulation (nuit, pluie, contre-jour) représentatives des conditions réelles de conduite.
- **Faible dégradation après quantification** : la perte de précision liée à la quantification INT8 doit rester inférieure à 3 % de mAP après calibration, préservant ainsi la qualité de détection sur les plateformes embarquées .
- **Maintenabilité et scalabilité** : l’architecture modulaire du pipeline doit faciliter les mises à jour futures, l’ajout de nouvelles classes d’objets et l’intégration de nouveaux capteurs sans remettre en cause la structure globale du système.

3.1.3 Synthèse des besoins

L’analyse des besoins met en évidence la nécessité de concevoir une solution combinant précision de détection, rapidité d’exécution, robustesse et faible complexité computationnelle. Ces exigences orientent directement les choix technologiques présentés dans les sections suivantes, tant au niveau matériel que logiciel.

3.2 Architecture globale du système

L'architecture proposée repose sur une chaîne de traitement modulaire organisée en quatre étapes successives, depuis la construction du dataset jusqu'au déploiement embarqué sur plateformes hétérogènes. La figure 3.1 illustre le pipeline complet du système ADAS.

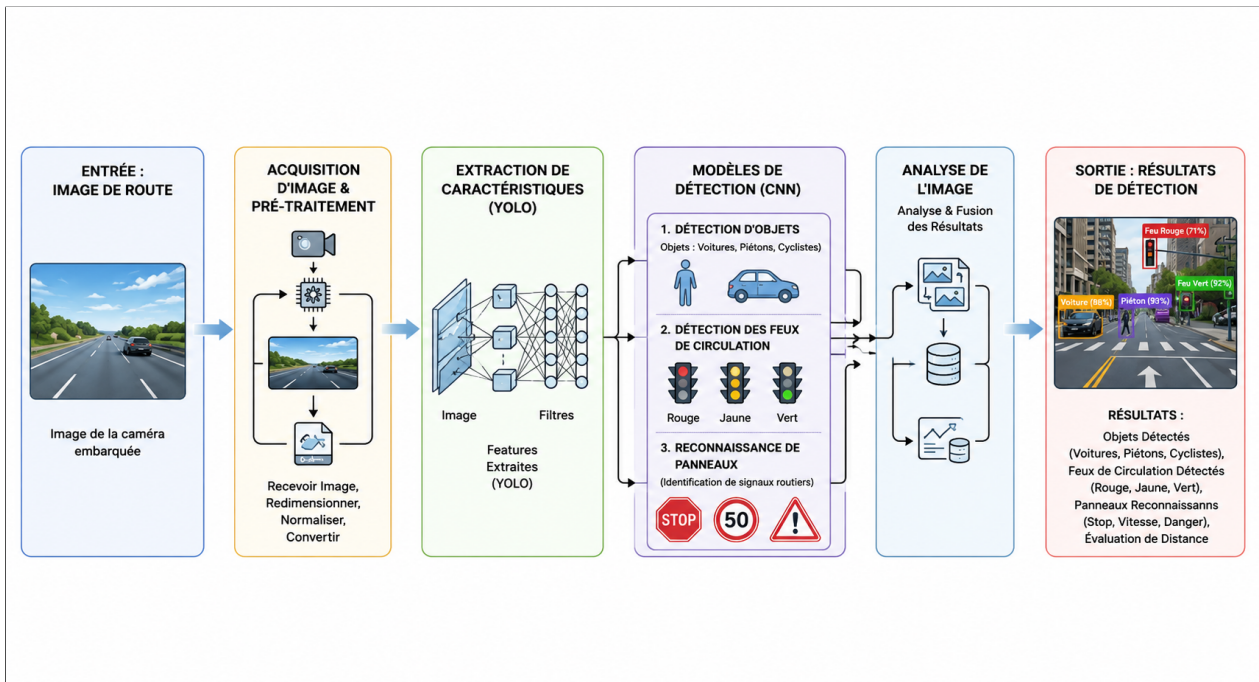


Figure 3.1 : Architecture globale du système ADAS : pipeline de traitement d'image et de détection d'objets

Le pipeline reçoit en entrée des images issues d'une caméra embarquée. La phase d'acquisition et de prétraitement assure le redimensionnement des images à 640x640 pixels, leur normalisation et la conversion de l'espace colorimétrique BGR vers RGB [4]. Le module d'extraction de caractéristiques basé sur l'architecture YOLOv8 [11] alimente trois sous-systèmes de détection parallèles : la détection d'objets (voitures, piétons, cyclistes), la détection des feux de circulation (rouge, jaune, vert) et la reconnaissance de panneaux routiers. Les résultats issus de ces trois modules sont ensuite fusionnés et présentés sous forme d'annotations visuelles en temps réel, incluant les boîtes englobantes et les scores de confiance associés à chaque détection.

3.3 Architectures partielles par étape

3.3.1 Étape 1 : construction et enrichissement du dataset

La première étape consiste à construire une base de données annotée adaptée aux besoins ADAS. Deux sources de données publiques complémentaires sont utilisées : GTSDb [2], spécialisé dans la détection de panneaux de signalisation routière, et Europe Signs, couvrant un large ensemble de panneaux de signalisation européens. L'accès à ces datasets est réalisé depuis l'environnement Kaggle Notebooks via l'API Roboflow, qui permet le téléchargement et la conversion automatique des annotations au format YOLO [11] sans manipulation manuelle. Les annotations issues des deux sources sont ensuite fusionnées et normalisées autour d'un référentiel de classes commun et fixe, garantissant une cohérence stricte tout au long du pipeline. Un modèle YOLOv8x pré-entraîné [11] est utilisé pour enrichir automatiquement le dataset par pseudo-labellisation, en générant des annotations supplémentaires pour les objets ADAS tels que les piétons, les véhicules et les feux de signalisation. Une étape de déduplication et de contrôle qualité est enfin appliquée afin d'éliminer les doublons et les annotations invalides, produisant une base de données cohérente et prête pour l'entraînement. La figure 3.2 illustre la construction.

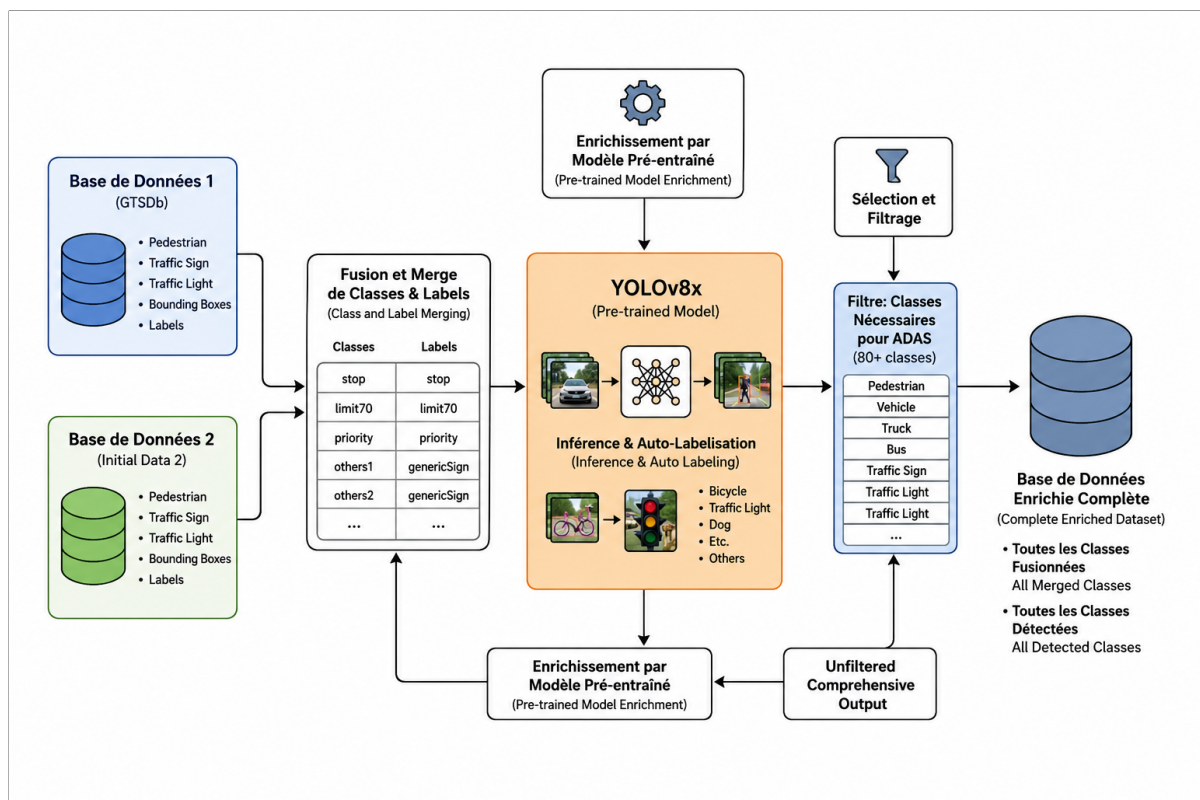


Figure 3.2 : Pipeline de construction et d'enrichissement du dataset ADAS

3.3.2 Étape 2 : entraînement indépendant Teacher et Student

La deuxième étape constitue la phase d'entraînement de référence, réalisée sans distillation. Le modèle teacher (YOLOv8m, haute complexité) et le modèle student de base (YOLOv8n, compact) sont entraînés indépendamment et en parallèle sur le dataset enrichi complet [11], comme illustré en figure 3.3.

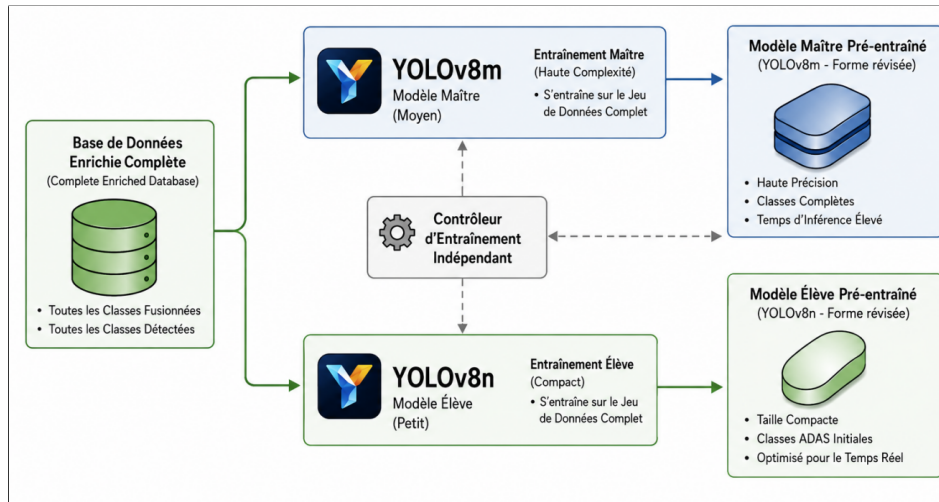


Figure 3.3 : Entraînement indépendant : *teacher* YOLOv8m et *student* baseline YOLOv8n

Un contrôleur d'entraînement indépendant supervise les deux processus simultanément. Le modèle teacher est optimisé pour atteindre une haute précision de détection sur l'ensemble des classes ADAS, quitte à présenter un temps d'inférence élevé. Le modèle student de base (YOLOv8n) est entraîné dans les mêmes conditions afin d'établir une référence de performance avant distillation. Ces deux modèles pré-entraînés sauvegardés sous forme de fichiers `best.pt` constituent les points de comparaison pour quantifier l'apport de la distillation à l'étape suivante.

3.3.3 Étape 3 : distillation des connaissances

La distillation de connaissances (Knowledge Distillation), introduite par Hinton et al. [6], consiste à transférer les connaissances acquises par un modèle complexe (teacher) vers un modèle léger (student). Plutôt que d'apprendre uniquement à partir des étiquettes dures (hard labels), le student apprend également à reproduire le comportement du teacher, ce qui lui permet d'atteindre des performances proches tout en conservant une faible complexité computationnelle [10].

Dans ce travail, deux canaux de transfert sont combinés :

- Distillation par les logits (Response-based KD) [6] : les distributions de probabilité softmax

produites par le teacher servent de cibles souples (soft labels) pour le student. Le paramètre de température T contrôle le lissage de ces distributions, tandis que le coefficient α ajuste l'équilibre entre la perte de supervision classique et la perte de distillation.

- Distillation par les caractéristiques intermédiaires (Feature-based KD / MGD) [7], [10] : les cartes de caractéristiques FPN intermédiaires du teacher supervisent celles du student, forçant ce dernier à apprendre des représentations internes plus riches, particulièrement utiles pour la détection de petits objets en conditions routières.

La figure 3.4 présente l'architecture de distillation utilisée dans ce projet.

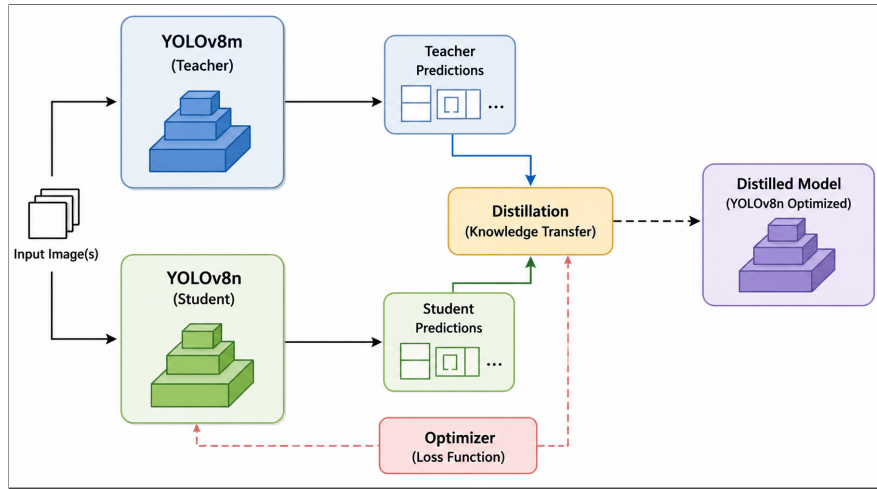


Figure 3.4 : Architecture de distillation des connaissances : YOLOv8m (*teacher*) $\rightarrow$ YOLOv8n (*student*)

La fonction de perte globale du student est définie comme une combinaison pondérée de la perte supervisée classique et de la perte de distillation [6] :

$$\mathcal{L}_{\text{total}} = \alpha \cdot \mathcal{L}_{\text{CE}} + (1 - \alpha) \cdot T^2 \cdot D_{\text{KL}}(p_{\text{teacher}}^T \| p_{\text{student}}^T) \quad (3.1)$$

où $\mathcal{L}_{\text{CE}}$ est l'entropie croisée avec les étiquettes réelles, D_{KL} la divergence de Kullback-Leibler entre les distributions adoucies, et T^2 un facteur de compensation de l'échelle des gradients.

3.3.4 Étape 4a : déploiement sur Jetson Nano (FP16 / TensorRT)

Le modèle student distillé est converti du format FP32 vers FP16 (Half Precision), puis compilé via TensorRT pour l'exécution sur la plateforme NVIDIA Jetson Nano [14]. La figure 3.5 illustre ce pipeline de déploiement.

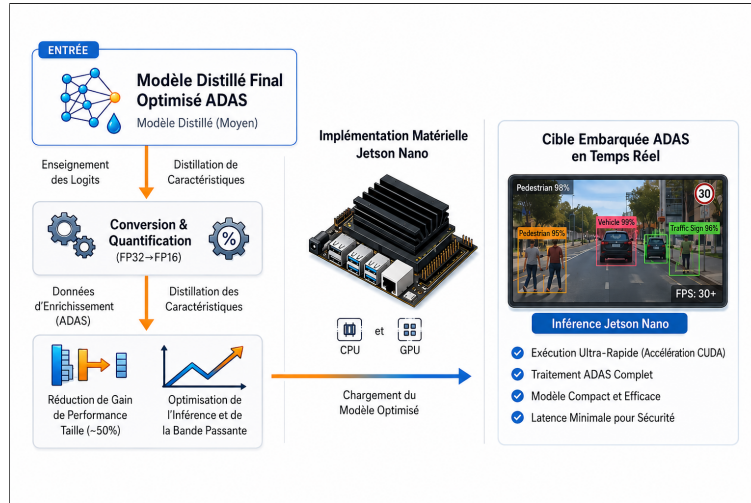


Figure 3.5 : Déploiement sur Jetson Nano avec optimisation FP16 / TensorRT

La quantification FP16 réduit la taille mémoire du modèle d'un facteur 2 par rapport au FP32, avec une dégradation de précision inférieure à 0,5 % de mAP. TensorRT réalise en outre plusieurs optimisations automatiques du graphe de calcul : fusion de couches, allocation optimisée de la mémoire GPU et génération d'instructions CUDA dédiées permettant d'exploiter pleinement les 128 cœurs Maxwell de la Jetson Nano. Le modèle optimisé atteint ainsi un débit supérieur à 30 FPS avec une latence inférieure à 40 ms, satisfaisant pleinement les exigences temps réel ADAS.

3.3.5 Étape 4b : déploiement sur FPGA ZedBoard (INT8 / Vitis AI)

Le modèle student distillé est quantifié au format INT8 puis déployé sur la ZedBoard Zynq-7020 [15] via le flux Vitis AI [16] et l'unité DPU (Deep Processing Unit). La figure 3.6 illustre ce déploiement.

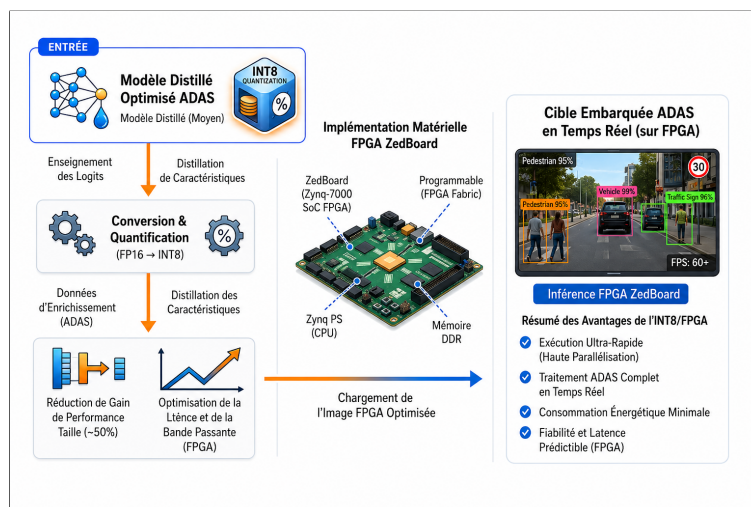


Figure 3.6 : Déploiement sur ZedBoard FPGA avec quantification INT8 / Vitis AI

L'architecture Zynq-7020 combine un processeur ARM double cur (partie PS) et une logique FPGA programmable de 85 000 cellules logiques (partie PL) [15]. La quantification INT8 réduit la taille mémoire d'un facteur 4 par rapport au FP32, avec une perte de précision inférieure à 1 % après calibration [10]. Le DPU instancié sur la partie PL assure une exécution hautement parallèle des couches convolutives, permettant d'atteindre plus de 60 FPS avec une consommation énergétique minimale [17]. Cette plateforme constitue ainsi la cible la plus efficace énergétiquement du projet.

3.4 Choix technologiques

Cette section présente les composants matériels et logiciels retenus pour la réalisation du projet, en justifiant chaque choix au regard des contraintes de performance, de déploiement et de compatibilité identifiées dans l'analyse des besoins.

3.4.1 Choix matériels

Pour le développement de notre projet, nous avons utilisé un ensemble de composants matériels pour répondre aux besoins de notre projet .

3.4.1.1 NVIDIA Jetson Nano

La plateforme NVIDIA Jetson Nano [14] est spécialement conçue pour les applications d'intelligence artificielle embarquée. Elle intègre un processeur ARM Cortex-A57 quadricur et un GPU CUDA de 128 curs Maxwell, permettant l'exécution parallèle des algorithmes de vision par ordinateur et des réseaux neuronaux profonds [18]. Son support natif de CUDA, cuDNN et TensorRT simplifie le déploiement de modèles optimisés et garantit une inférence rapide, locale et fiable. Elle fonctionne sous Ubuntu 18.04 LTS (JetPack SDK), la distribution officiellement supportée par NVIDIA pour cette plateforme, intégrant nativement les drivers CUDA, cuDNN et TensorRT nécessaires au déploiement des modèles de détection. Sa consommation comprise entre 5 et 10 W en fait une solution particulièrement adaptée aux systèmes embarqués à ressources limitées. comme illustré dans la figure 3.7.

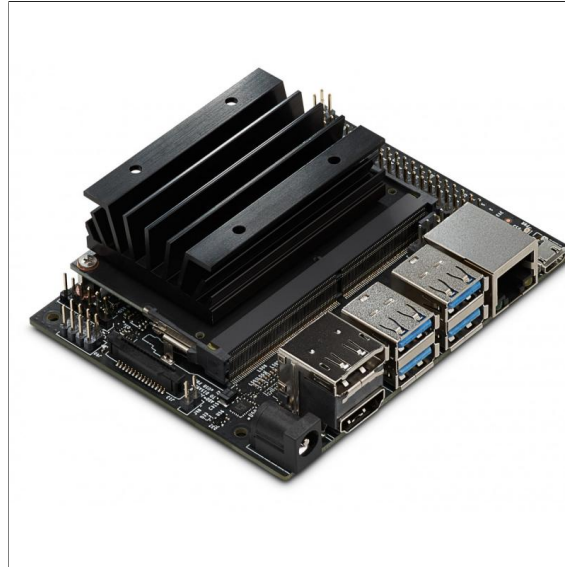


Figure 3.7 : Carte NVIDIA Jetson Nano

3.4.1.2 ZedBoard Xilinx Zynq-7020

La ZedBoard [15] est une plateforme de prototypage basée sur le composant Xilinx Zynq-7020, qui associe dans une même architecture un processeur ARM Cortex-A9 double cur (partie PS) et une logique FPGA reconfigurable de 85 000 cellules logiques (partie PL). Cette architecture hétérogène offre une grande flexibilité pour l'accélération matérielle d'opérations critiques telles que les convolutions et le pooling, directement implémentées sur la fabric FPGA [17]. La présence d'interfaces de communication variées (GPIO, UART, SPI, I2C, Ethernet) facilite l'intégration de capteurs complémentaires dans une architecture embarquée complexe. comme illustré dans la figure 3.8.

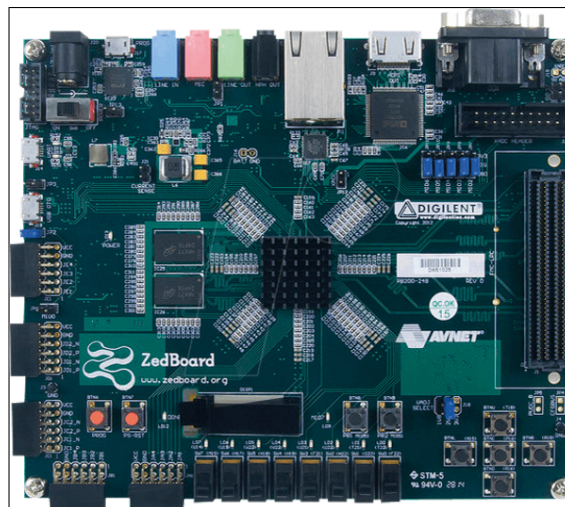


Figure 3.8 : carte FPGA zedboard

3.4.1.3 Caméra Raspberry Pi Camera Module V2

L'acquisition des images est assurée par la Raspberry Pi Camera Module V2, équipée du capteur Sony IMX219 8 mégapixels. Ce module communique via une interface CSI (Camera Serial Interface), garantissant un transfert bas-latence vers le processeur embarqué, contrairement aux solutions USB plus gourmandes en ressources. Sa compatibilité native avec OpenCV [4] simplifie l'intégration dans le pipeline de capture et de prétraitement temps réel.

3.4.2 Choix logiciels

Dans cette partie, nous allons présenter les différents logiciels et outils utilisés dans le cadre de ce projet.

3.4.2.1 Système d'exploitation

Deux environnements Linux sont utilisés selon la plateforme cible. Le PC de développement fonctionne sous Ubuntu 24.04 LTS, offrant un environnement moderne et stable pour l'installation des frameworks PyTorch, Ultralytics et des outils de compilation. La Jetson Nano fonctionne sous Ubuntu 18.04 LTS via le JetPack SDK de NVIDIA [14], seule version officiellement supportée par les drivers CUDA 10.2, cuDNN 8 et TensorRT 8 nécessaires au déploiement des modèles optimisés.

3.4.2.2 JupyterLab

JupyterLab constitue l'environnement de développement interactif principal, utilisé aussi bien sur le PC de développement que sur la Jetson Nano via accès SSH. Il permet le prototypage rapide, la visualisation interactive des courbes d'entraînement et de validation, ainsi que l'exécution pas à pas des scripts d'évaluation et de benchmarking. Son interface basée sur navigateur simplifie le développement à distance sur les plateformes embarquées.

3.4.2.3 Google Colab et Kaggle Notebooks

Google Colab offre un accès cloud à des GPU puissants (T4, A100), ce qui accélère significativement les phases d'entraînement des modèles teacher et student. Son intégration avec Google Drive facilite le stockage des datasets, des poids de modèles et des résultats expérimentaux. Kaggle Notebooks constitue un environnement complémentaire, offrant un accès natif aux datasets publics (GTSDb + Europe Signs) via l'API Kaggle sans phase complexe de téléchargement, ainsi que des ressources

GPU et TPU pour l'expérimentation parallèle.

3.4.2.4 Python

Python constitue le langage principal du projet. Sa syntaxe concise, sa richesse en bibliothèques spécialisées dans l'apprentissage profond et le traitement d'image, ainsi que son interopérabilité avec CUDA et TensorRT en font le choix naturel pour l'ensemble des phases de développement, d'expérimentation et d'évaluation.

3.4.2.5 VHDL

Le langage VHDL (VHSIC Hardware Description Language) est utilisé pour la modélisation, la simulation et la validation des blocs matériels reconfigurables implémentés sur la partie FPGA de la ZedBoard. Il permet une conception rigoureuse à différents niveaux d'abstraction comportemental et structurel ainsi qu'une vérification fonctionnelle précise par simulation avant déploiement sur le matériel physique.

3.4.2.6 Vivado et Vitis HLS

Vivado est l'environnement de développement FPGA de Xilinx [16]. Il assure la synthèse logique, le placement-routage et la génération du bitstream pour le Zynq-7020. Vitis HLS permet la synthèse de haut niveau à partir de code C/C++, facilitant l'intégration de blocs d'accélération matérielle personnalisés dans la chaîne de traitement FPGA.

3.4.2.7 SSH

Le protocole SSH (Secure Shell) est utilisé pour l'accès distant sécurisé aux plateformes embarquées Jetson Nano et ZedBoard. Il permet l'administration distante, l'exécution de commandes système, le transfert de fichiers et la supervision des performances en temps réel, sans nécessiter d'accès physique direct aux cartes.

3.4.3 Bibliothèques Python utilisées

- **ultralytics** : bibliothèque principale encapsulant l'architecture YOLOv8 [11]. Elle fournit une API unifiée pour l'entraînement, la validation, l'export multi-format (ONNX, TensorRT) et l'inférence. Elle est utilisée dans les phases d'entraînement, de distillation et de déploiement.
- **roboflow** : bibliothèque Python officielle de la plateforme Roboflow, utilisée pour le téléchargement

et la conversion automatique des datasets annotés au format YOLO. Elle simplifie l'accès aux datasets ADAS spécialisés via une API clé en main, sans manipulation manuelle des fichiers d'annotations.

- **kaggle** : bibliothèque officielle de l'API Kaggle, utilisée pour le téléchargement automatisé des datasets publics (COCO, BDD100K, GTSDb) directement depuis les environnements Colab et Jupyter, via la commande `kaggle datasets download -d <dataset>`.
- **torch / torchvision** : framework PyTorch [6] utilisé pour la définition des fonctions de perte personnalisées de distillation, la gestion des optimiseurs et l'ensemble des opérations d'apprentissage profond.
- **opencv-python** : bibliothèque de vision par ordinateur [4] utilisée pour la lecture des flux vidéo, le redimensionnement des images, la normalisation et la conversion des espaces colorimétriques BGR/RGB en entrée du pipeline.
- **albumentations** : bibliothèque d'augmentation de données appliquée lors de la phase de construction du dataset. Elle permet d'enrichir artificiellement les données d'entraînement par rotation, variation de luminosité, zoom, bruit et modification de contraste.
- **numpy** : bibliothèque de calcul numérique utilisée pour la manipulation des tenseurs, des matrices de prédictions et des résultats de benchmarking tout au long du projet.
- **matplotlib / seaborn** : bibliothèques de visualisation scientifique utilisées pour tracer les courbes de perte, les courbes mAP, et les graphiques comparatifs de performances entre plateformes.
- **onnx / onnxruntime** : utilisées pour l'export du modèle au format ONNX et son exécution sur différentes plateformes, assurant l'interopérabilité entre l'environnement d'entraînement PyTorch et les cibles de déploiement embarquées.
- **tensorrt** : bibliothèque NVIDIA utilisée pour la compilation et l'optimisation du modèle exporté sur Jetson Nano [14]. Elle réalise la fusion de couches, l'allocation mémoire GPU optimisée et la génération du moteur d'inférence FP16.
- **vai_q_pytorch** : outil de quantification Vitis AI [16] utilisé pour la conversion du modèle en INT8 avant déploiement sur la ZedBoard FPGA, avec calibration sur un sous-ensemble représentatif du dataset.
- **pyyaml** : bibliothèque de lecture et d'écriture des fichiers de configuration YAML utilisés par Ultralytics YOLOv8 pour la définition des datasets, des hyperparamètres et des architectures de modèles.

Conclusion

Ce chapitre a présenté l'organisation générale du système ainsi que les différents choix effectués pour sa mise en place. L'ensemble des décisions prises vise à répondre aux exigences du projet tout en tenant compte des contraintes identifiées. La réalisation concrète de ce système sera présentée dans le chapitre suivant.

Réalisation

Plan

1	Choix techniques	44
2	Plateformes embarquées pour l'inférence IA	44
3	Travail réalisé	47
4	Planning réel du projet	47

Introduction

Introduction partielle, qui annonce le contenu.

4.1 Choix techniques

4.2 Plateformes embarquées pour l'inférence IA

4.2.1 Critères de sélection

Le choix d'une plateforme matérielle pour le déploiement d'un modèle de détection d'objets en contexte ADAS doit prendre en compte plusieurs critères critiques [18] :

- **Débit d'inférence** (FPS) : capacité à traiter les trames vidéo en temps réel.
- **Latence** : délai de traitement bout en bout, critique pour la sécurité.
- **Consommation énergétique** (W) : contrainte forte en environnement embarqué.
- **Efficacité énergétique** (TOPS/W) : performance par watt consommé.
- **Encombrement et coût** : volume, poids et prix unitaire.
- **Écosystème logiciel** : disponibilité des frameworks (TensorRT, OpenVINO, Vitis AI).

4.2.2 GPU : NVIDIA Jetson Nano

Le Jetson Nano est une plateforme GPU embarquée développée par NVIDIA, spécifiquement conçue pour les applications d'intelligence artificielle à la périphérie du réseau (*edge AI*) [14]. Il intègre un GPU Maxwell de 128 cœurs CUDA et un processeur ARM Cortex-A57 quadricur.

Caractéristiques techniques :

- GPU : 128 cœurs CUDA (architecture Maxwell)
- CPU : ARM Cortex-A57 quadricur @ 1.43 GHz
- Mémoire : 4 Go LPDDR4 partagée CPU/GPU
- Puissance de calcul : 472 GFLOPS (FP16)
- Consommation : 5–10 W (modes économie d'énergie configurables)
- Interfaces : GPIO, CSI, MIPI, USB 3.0, Gigabit Ethernet

Écosystème logiciel : Le Jetson Nano bénéficie du *JetPack SDK*, comprenant TensorRT pour l'optimisation d'inférence, CUDA, cuDNN et les frameworks PyTorch/TensorFlow. TensorRT permet notamment la quantification INT8 ou FLOAT16 et la fusion de couches pour maximiser les

performances. comme illustré dans la figure 4.2.

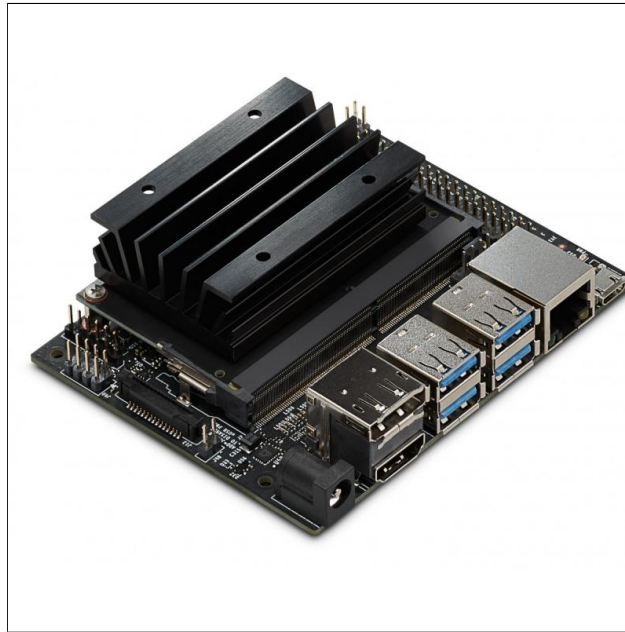


Figure 4.1 : carte jetson nano

4.2.3 FPGA : ZedBoard (Xilinx Zynq-7020)

Le ZedBoard est une carte de développement basée sur le SoC Xilinx Zynq-7020, combinant un processeur ARM Cortex-A9 dual-core avec une structure logique FPGA reconfigurable [17]. Cette architecture hétérogène (*Zynq APSoC*) offre une flexibilité maximale pour l'implémentation d'accélérateurs matériels dédiés.

Caractéristiques techniques :

- PL (*Programmable Logic*) : 85K cellules logiques, 220 DSP48E1, 560 Ko BRAM
- PS (*Processing System*) : ARM Cortex-A9 dual-core @ 667 MHz
- Mémoire : 512 Mo DDR3 + 256 Mo QSPI Flash
- Interfaces : HDMI, USB OTG, Gigabit Ethernet, connecteurs PMOD
- Consommation : 3–8 W selon la configuration

Flux de conception : Le déploiement d'un réseau de neurones sur FPGA requiert un flux spécialisé. Xilinx propose la suite *Vitis AI*, comprenant le compilateur DPU (*Deep Processing Unit*), un overlay matériel prédéfini pour l'inférence de CNNs, et des outils de quantification et de pruning.

Avantages spécifiques au FPGA :

- Parallélisme spatial reconfigurable à la granularité bit près.
- Efficacité énergétique supérieure pour des charges de travail fixes.



Figure 4.2 : carte FPGA zedboard

Le PC équipé d'un GPU NVIDIA (série RTX/GTX) constitue la plateforme de référence pour l'entraînement et l'optimisation des modèles. Il permet d'exploiter pleinement les capacités de CUDA et TensorRT sans contraintes de ressources, servant de référence pour l'évaluation des performances avant déploiement embarqué.

4.2.5 Comparaison des plateformes cibles

Critère	PC (GPU dédié)	Jetson Nano	ZedBoard (FPGA)
Performance brute	Très élevée	Moyenne	Élevée (tâches fixes)
Consommation (W)	150–350 W	5–10 W	3–8 W
Efficacité (TOPS/W)	Faible	Moyenne	Élevée
Latence déterministe	Non	Partielle	Oui
Flexibilité logicielle	Très élevée	Élevée	Faible
Complexité de déploiement	Faible	Moyenne	Très élevée
Coût	Élevé	Modéré	Modéré
Adapté au prototypage	Oui	Oui	Non (long. cycle)
Adapté à la production	Non	Oui	Oui

Tableau 4.1 : Comparaison des plateformes cibles pour le déploiement du modèle ADAS

4.3 Travail réalisé

4.4 Planning réel du projet

Conclusion

Conclusion partielle ayant pour objectif de synthétiser le chapitre et d'annoncer le chapitre suivant.

Conclusion générale

Rappel du contexte et de la problématique.

Brève récapitulation du travail réalisé et de la solution proposée.

La taille de la conclusion doit être réduite, une page de texte tout au plus. Il est important de souligner que la conclusion ne comporte pas de nouveaux résultats ni de nouvelles interprétations.

Le plus souvent, la conclusion comporte :

[label=•,font=]

- un résumé très rapide du corps du texte ;
- un rappel des objectifs du projet ;
- un bilan professionnel qui indique clairement les objectifs annoncés dans l'introduction et en particulier ceux qui n'ont pu être atteints. Il présente et synthétise les conclusions partielles ;
- un bilan personnel qui décrit les principales leçons que vous tirez de cette expérience sur le plan humain ;
- les limites et les perspectives du travail effectué.

Bibliographie

- [1] A. ZIEBINSKI, R. CUPEK, D. GRZECHCA et L. CHRUSZCZYK, « Review of Advanced Driver Assistance Systems (ADAS), » *AIP Conference Proceedings*, t. 1906, n° 1, 2017, Consulté le 5 avril 2026. DOI : 10.1063/1.5012394 adresse : <https://doi.org/10.1063/1.5012394>
- [2] K. BENGLER, K. DIETMAYER, B. FARBER, M. MAURER, C. STILLER et H. WINNER, « Three Decades of Driver Assistance Systems : Review and Future Perspectives, » *IEEE Intelligent Transportation Systems Magazine*, t. 6, n° 4, p. 6-22, 2014, Consulté le 5 avril 2026. DOI : 10.1109/MITS.2014.2336271 adresse : <https://doi.org/10.1109/MITS.2014.2336271>
- [3] VALEO, *Solutions ADAS complètes*, Consulté le 22 mai 2026, 2024. adresse : <https://www.valeo.com/fr/catalogue/cda/solutions-adas-completes/>
- [4] R. SZELISKI, *Computer Vision : Algorithms and Applications*, 2^e éd. Springer, 2022, Consulté le 5 avril 2026, ISBN : 978-3-030-34372-9. adresse : <https://szeliski.org/Book/>
- [5] J. REDMON, S. DIVVALA, R. GIRSHICK et A. FARHADI, « You Only Look Once : Unified, Real-Time Object Detection, » in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Consulté le 5 avril 2026, 2016, p. 779-788. DOI : 10.1109/CVPR.2016.91 adresse : <https://arxiv.org/abs/1506.02640>
- [6] G. HINTON, O. VINYALS et J. DEAN, « Distilling the Knowledge in a Neural Network, » *arXiv preprint arXiv :1503.02531*, 2015, NIPS 2014 Deep Learning Workshop. Consulté le 5 avril 2026. adresse : <https://arxiv.org/abs/1503.02531>
- [7] A. ROMERO, N. BALLAS, S. E. KAHOU, A. CHASSANG, C. GATTA et Y. BENGIO, « FitNets : Hints for Thin Deep Nets, » in *Proceedings of the International Conference on Learning Representations (ICLR)*, Consulté le 5 avril 2026, 2015. adresse : <https://arxiv.org/abs/1412.6550>
- [8] S. ZAGORUYKO et N. KOMODAKIS, « Paying More Attention to Attention : Improving the Performance of Convolutional Neural Networks via Attention Transfer, » in *Proceedings of the International Conference on Learning Representations (ICLR)*, Consulté le 5 avril 2026, 2017. adresse : <https://arxiv.org/abs/1612.03928>

- [9] W. PARK, D. KIM, Y. LU et M. CHO, « Relational Knowledge Distillation, » in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Consulté le 5 avril 2026, 2019, p. 3962-3971. DOI : 10.1109/CVPR.2019.00409 adresse : <https://arxiv.org/abs/1904.05068>
- [10] J. GOU, B. YU, S. J. MAYBANK et D. TAO, « Knowledge Distillation : A Survey, » *International Journal of Computer Vision*, t. 129, n° 6, p. 1789-1819, 2021, Consulté le 5 avril 2026. DOI : 10.1007/s11263-021-01453-z adresse : <https://arxiv.org/abs/2006.05525>
- [11] G. JOCHER, A. CHAURASIA et J. QIU, *Ultralytics YOLOv8*, version 8.0.0, Consulté le 5 avril 2026, 2023. adresse : <https://github.com/ultralytics/ultralytics>
- [12] S. REN, K. HE, R. GIRSHICK et J. SUN, « Faster R-CNN : Towards Real-Time Object Detection with Region Proposal Networks, » in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015, p. 91-99. adresse : <https://arxiv.org/abs/1506.01497>
- [13] W. LIU et al., « SSD : Single Shot MultiBox Detector, » in *European Conference on Computer Vision (ECCV)*, 2016, p. 21-37. adresse : <https://arxiv.org/abs/1512.02325>
- [14] NVIDIA CORPORATION, *Jetson Nano Developer Kit – User Guide*, Consulté le 5 avril 2026, 2019. adresse : <https://developer.nvidia.com/embedded/learn/get-started-jetson-nano-devkit>
- [15] AVNET INC., *ZedBoard : Zynq-7000 Development Board*, Consulté le 5 avril 2026, 2023. adresse : <https://www.avnet.com/wps/portal/us/products/avnet-boards/avnet-board-families/zedboard/>
- [16] AMD XILINX, *Vitis AI – Optimized AI Inference for Zynq and Alveo*, Consulté le 5 avril 2026, 2023. adresse : <https://www.xilinx.com/products/design-tools/vitis/vitis-ai.html>
- [17] E. NURVITADHI, D. SHEFFIELD, J. SIM, A. MISHRA, G. VENKATESH et D. MARR, « Accelerating Binarized Neural Networks : Comparison of FPGA, CPU, GPU, and ASIC, » in *International Conference on Field-Programmable Technology (FPT)*, Consulté le 5 avril 2026, 2016, p. 77-84. DOI : 10.1109/FPT.2016.7929192 adresse : <https://doi.org/10.1109/FPT.2016.7929192>
- [18] S. DENG, H. ZHAO, W. FANG, J. YIN, S. DUSTDAR et A. Y. ZOMAYA, « Edge Intelligence : The Confluence of Edge Computing and Artificial Intelligence, » *IEEE Internet of Things Journal*, t. 7, n° 8, p. 7457-7469, 2020, Consulté le 5 avril 2026. DOI : 10.1109/JIOT.2020.2984887 adresse : <https://doi.org/10.1109/JIOT.2020.2984887>

Annexes

Annexe 1. Exemple d'annexe

Les chapitres doivent présenter l'essentiel du travail. Certaines informations-trop détaillées ou constituant un complément d'information pour toute personne qui désire mieux comprendre ou refaire une expérience décrite dans le document- peuvent être mises au niveau des annexes. Les annexes, **placées après la bibliographie**, doivent donc être numérotées avec des titres (Annexe1, Annexe2, etc.).

Le tableau annexe 1.1 présente un exemple d'un tableau dans l'annexe.

Tableau annexe 1.1 : Exemple tableau dans l'annexe

0	0
1	1
2	2
3	3
4	4

Annexe 2. Entreprise

La figure annexe 2.1 présente le logo entreprise.

Figure annexe 2.1 : Logo d'entreprise

... Exemple ici ... :
:

Résumé

Mettez le résumé en français ici... Mettez le résumé en français ici... Mettez le résumé en français ici... Mettez le résumé en français ici... Merci de ne pas dépasser les dix lignes. Mettez le résumé en français ici, merci de ne pas dépasser les dix lignes. Mettez le résumé en français ici, merci de ne pas dépasser les dix lignes. Mettez le résumé en français ici, merci de ne pas dépasser les dix lignes. Mettez le résumé en français ici, merci de ne pas dépasser les dix lignes. Mettez le résumé en français ici, merci de ne pas dépasser les dix lignes. Mettez le résumé en français ici, merci de ne pas dépasser les dix lignes. Mettez le résumé en français ici, merci de ne pas dépasser les dix lignes. Mettez le résumé en français ici, merci de ne pas dépasser les dix lignes.

Mots clés : Merci de ne pas dépasser les cinq mots

Abstract

Put the English abstract here, put the English abstract here, put the English abstract here, put the English abstract here, put the English abstract here... Please don't exceed ten lines, Please don't exceed ten lines, Please don't exceed ten lines, Please don't exceed ten lines. Put the English abstract here, please don't exceed ten lines. Put the English abstract here, please don't exceed ten lines. Put the English abstract here, please don't exceed ten lines. Put the English abstract here, please don't exceed ten lines. Put the English abstract here, please don't exceed ten lines. Put the English abstract here, please don't exceed ten lines. Put the English abstract here, please don't exceed ten lines. Put the English abstract here, please don't exceed ten lines. Put the English abstract here, please don't exceed ten lines.

Keywords : Please don't use more than five keywords